Tasks on docker challenges

Question 1: Create a Customized Docker Image Using a Dockerfile

Implementation Steps

1. Connected to the EC2 instance using SSH.
2. Installed Docker and started the Docker service.
3. Created a project directory for the application.
4. Created a Dockerfile to define the application environment, required files, dependencies, and startup command.

```
FROM amazonlinux:latest
MAINTAINER ="Vamshi"
RUN dnf install -y nginx && dnf clean all
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

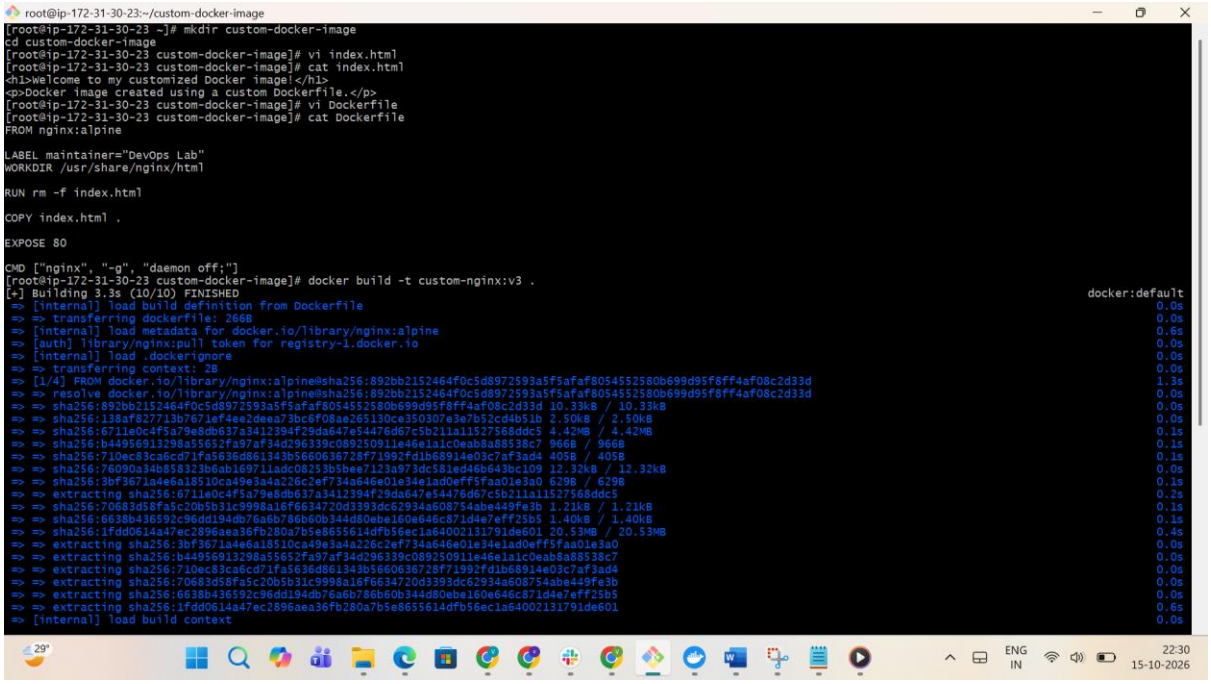
5. Selected an appropriate base image for the application.
6. Copied the application files into the Docker image using the COPY instruction.
7. Configured the required working directory and exposed the application port.
8. Built the customized Docker image using:

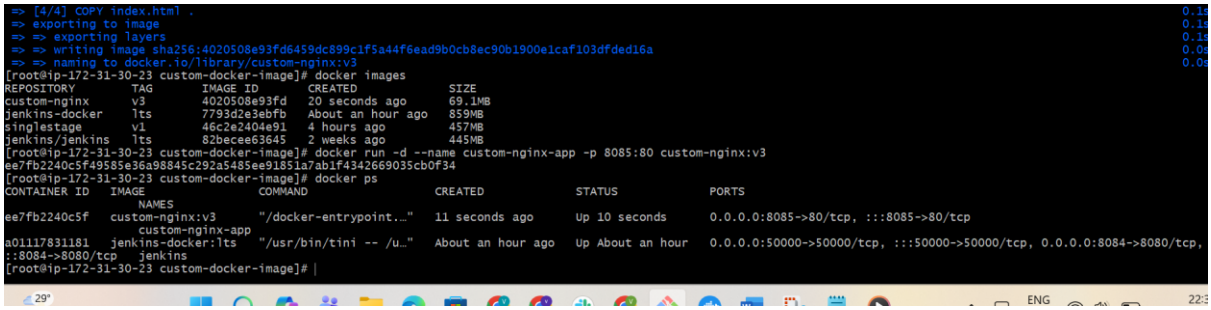
   docker build -t custom-nginx:v3 .
9. Verified that the image was created successfully:

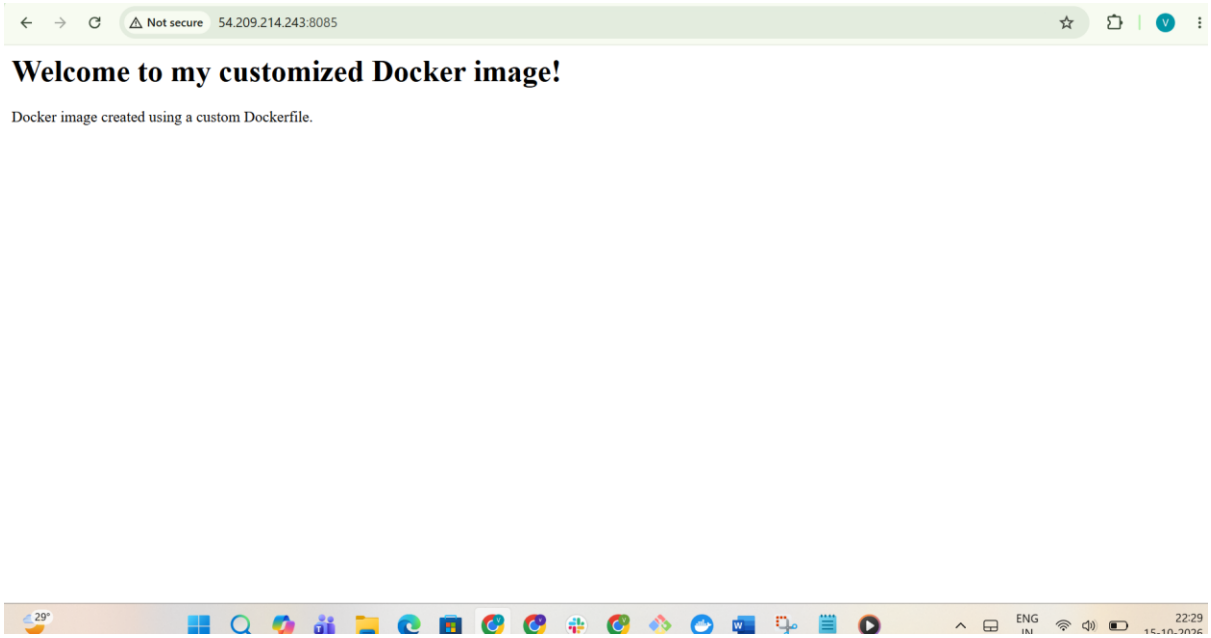
   docker images
10. Created and started a container from the customized image:

    docker run -d --name custom-app-container -p 8085:80 custom-nginx:v3
11. Verified the container status:

    docker ps
12. Tested the application using the appropriate browser URL or curl command.







Question 2: Push the Customized Docker Image to Docker Hub

Implementation Steps

1. Created a Docker Hub account and a repository for the customized image.

2. Logged in to Docker Hub from the EC2 instance:

   docker login

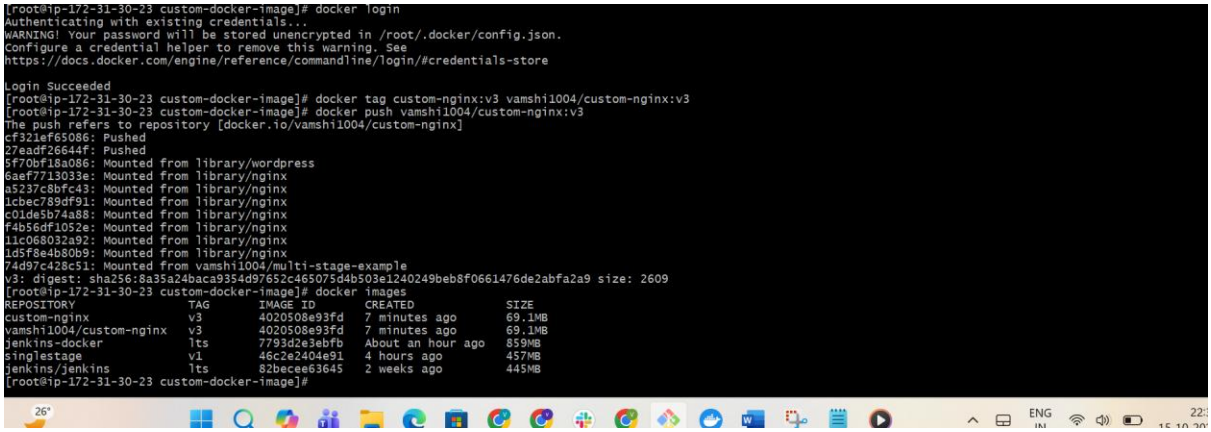
3. Tagged the customized image with the Docker Hub username and repository name:

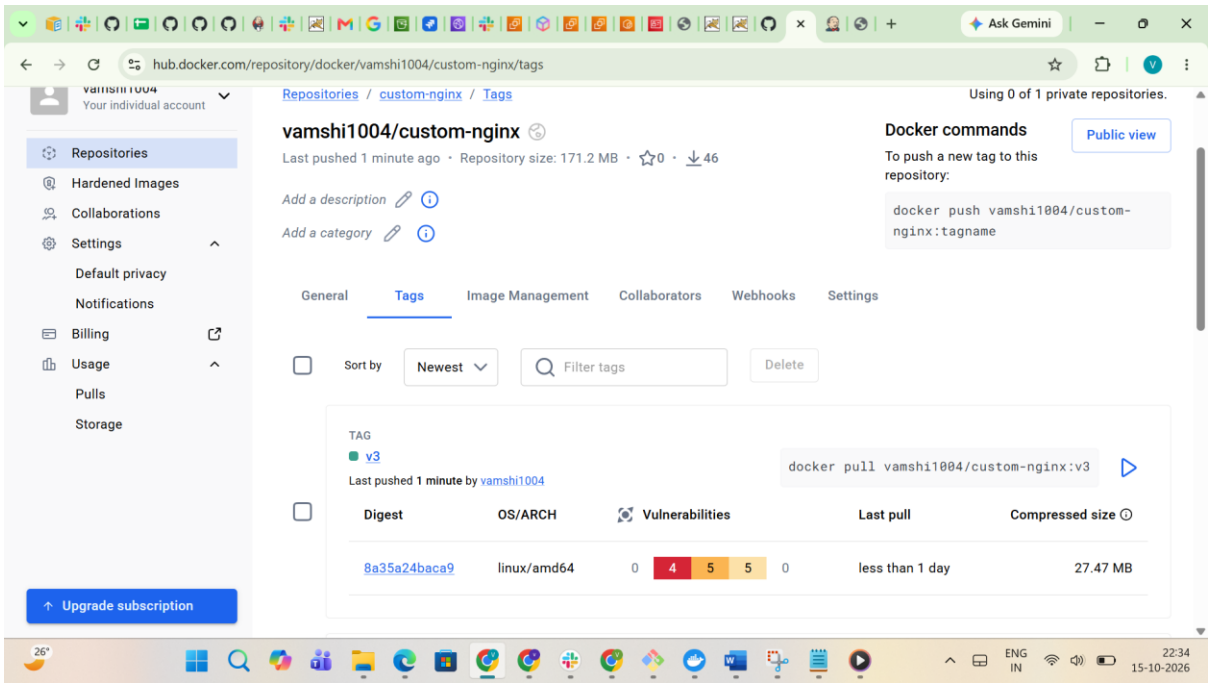
   docker tag custom-nginx:v3 DOCKERHUB_USERNAME/custom-nginx:v3

4. Pushed the image to Docker Hub:

   docker push DOCKERHUB_USERNAME/custom-nginx:v3

5. Verified the image in the Docker Hub repository.





Question 3: Push the Same Docker Image to Amazon ECR

Implementation Steps

1. Opened the AWS Management Console and navigated to Amazon Elastic Container Registry (ECR).

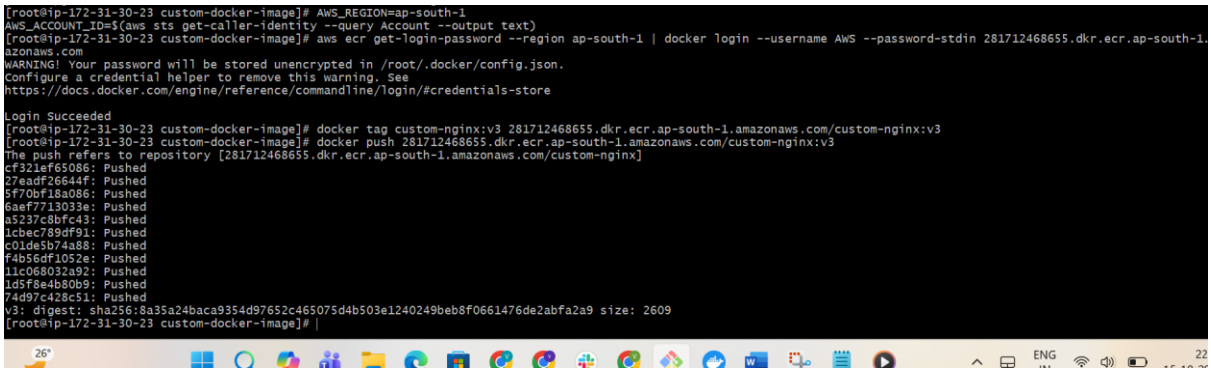
2. Created an ECR repository named custom-app in the ap-south-1 region.

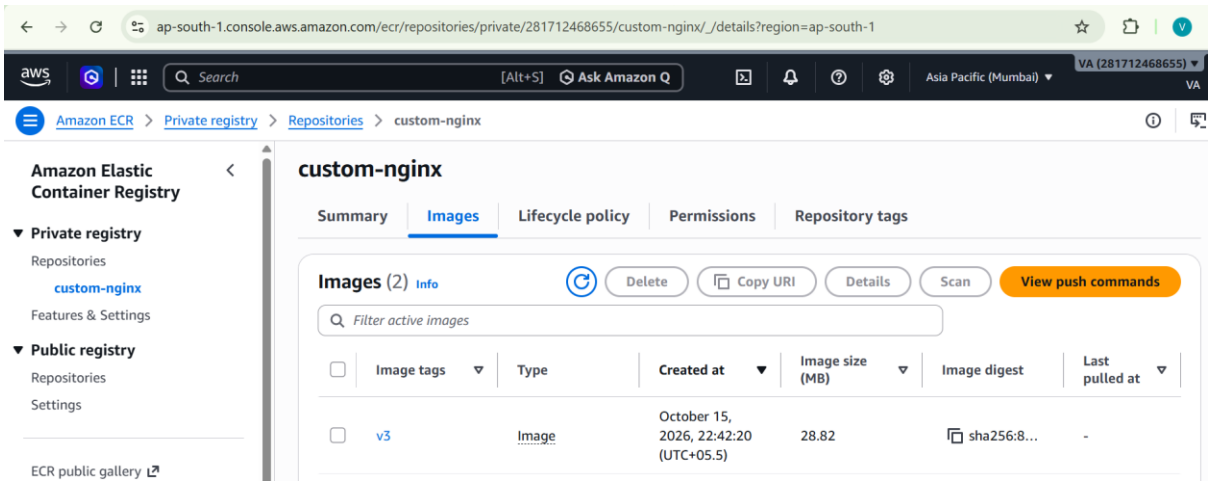
3. Verified that the AWS CLI was installed and that valid AWS credentials were configured.
4. Retrieved an authentication token and logged in to Amazon ECR:

   aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 281712468655.dkr.ecr.ap-south-1.amazonaws.com
5. Tagged the existing customized Docker image for Amazon ECR:

   docker tag custom-nginx:v3 281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx:v3
6. Pushed the same image to the ECR repository:

   docker push 281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx:v3
7. Opened the ECR repository in the AWS Console.
8. Verified that the image and its tag were available in the repository.





Question 4: Provision an EC2 Instance Using Terraform and Install Jenkins

Implementation Steps

1. Verified that Terraform and the AWS CLI were installed.
2. Configured AWS credentials or an appropriate IAM role for Terraform.
3. Created a project directory for the Terraform configuration:

   mkdir terraform-jenkins

cd terraform-jenkins

4. Created the Terraform configuration file named main.tf.
5. Configured the AWS provider with the required region, ap-south-1.
6. Added a data source to identify a suitable Amazon Linux AMI.
7. Defined an EC2 security group with the required inbound rules, including SSH and the Jenkins web port.
8. Defined an EC2 instance resource with the required AMI, instance type, key pair, security group, and user data.
9. Configured the user-data script to install Java, install Jenkins, enable the Jenkins service, and start Jenkins.
10. Initialized the Terraform working directory:

    terraform init

11. Validated the Terraform configuration:

    terraform validate

12. Reviewed the planned AWS resources:

    terraform plan

13. Provisioned the EC2 instance:

    terraform apply

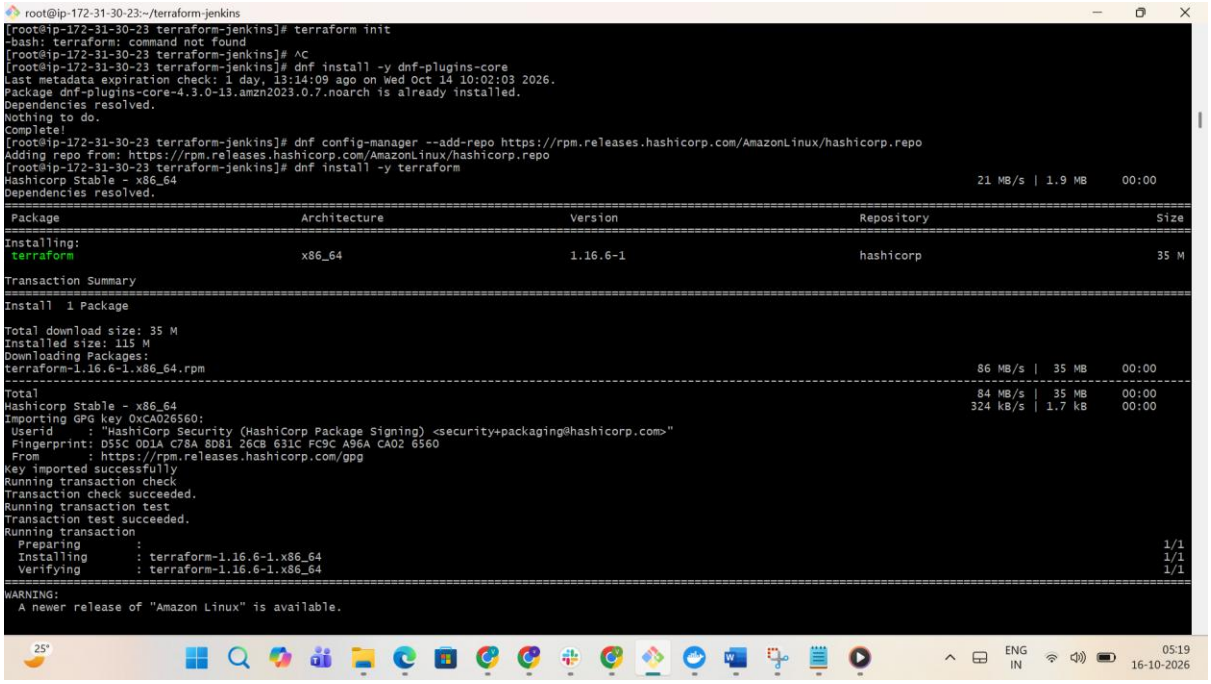
14. Confirmed the deployment when prompted.
15. Verified that the EC2 instance was created successfully in the AWS Console.
16. Connected to the instance using SSH and checked the Jenkins service:

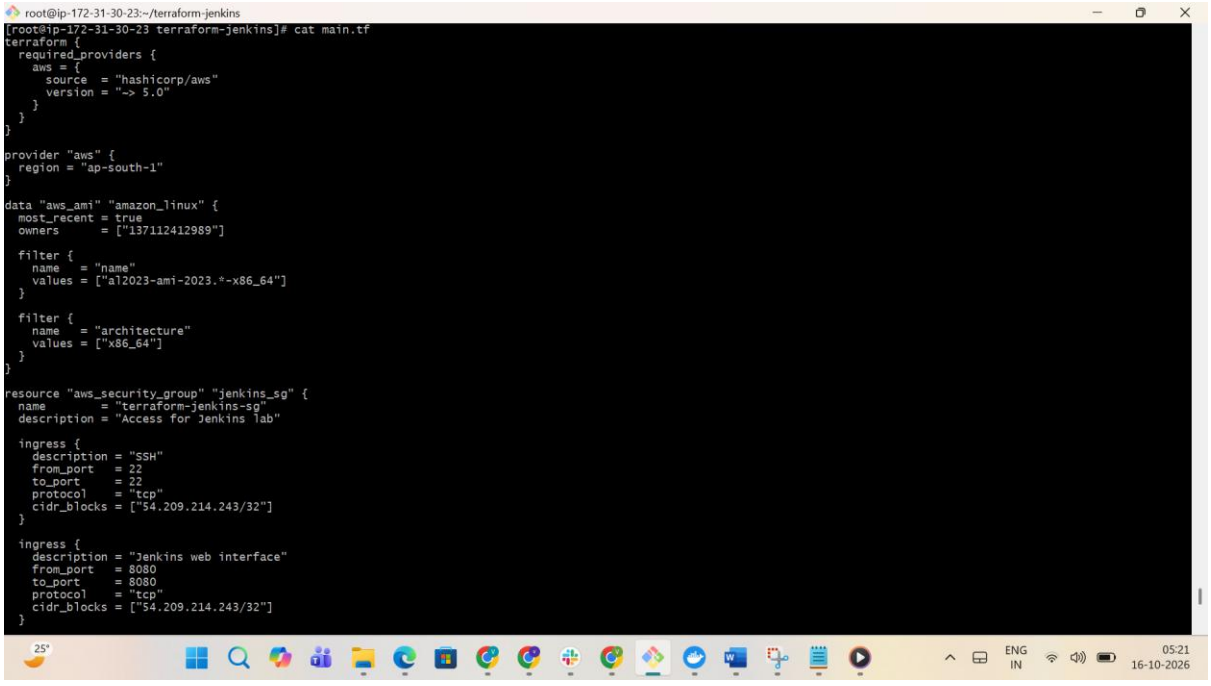
    sudo systemctl status jenkins

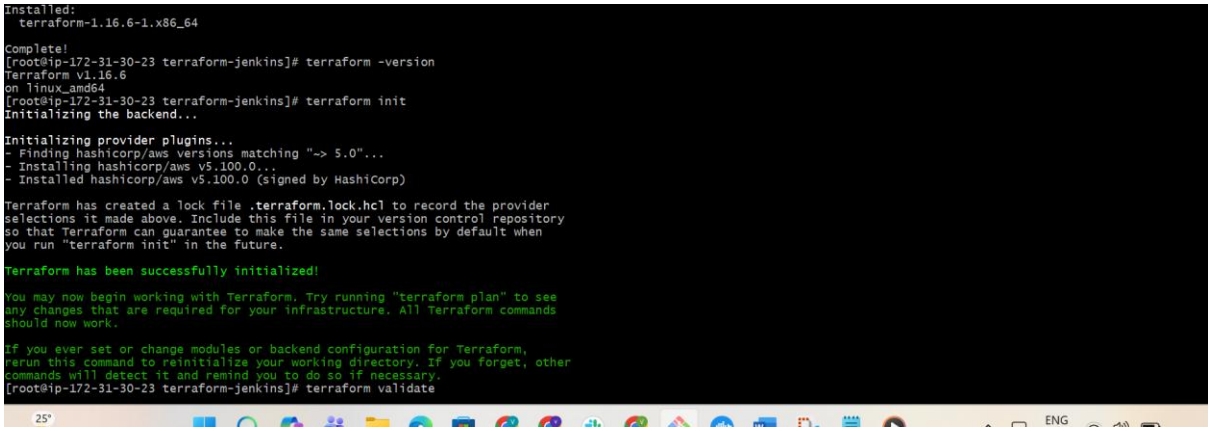
17. Retrieved the initial Jenkins administrator password:

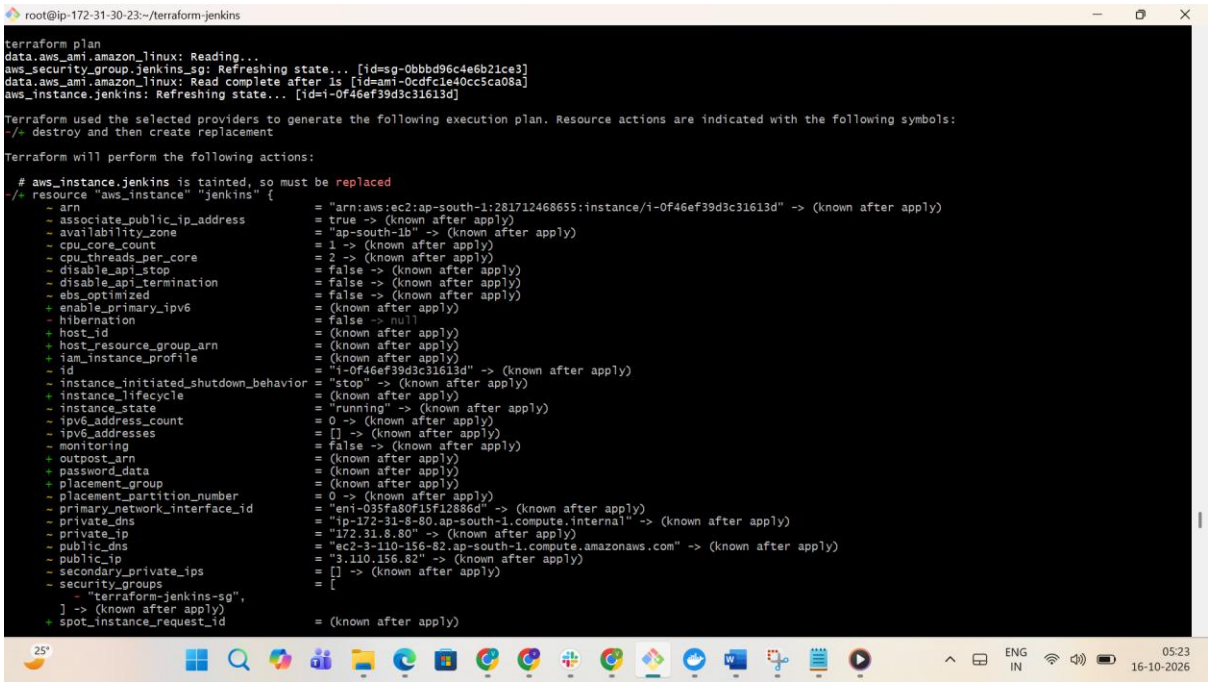
    sudo cat /var/lib/jenkins/secrets/initialAdminPassword

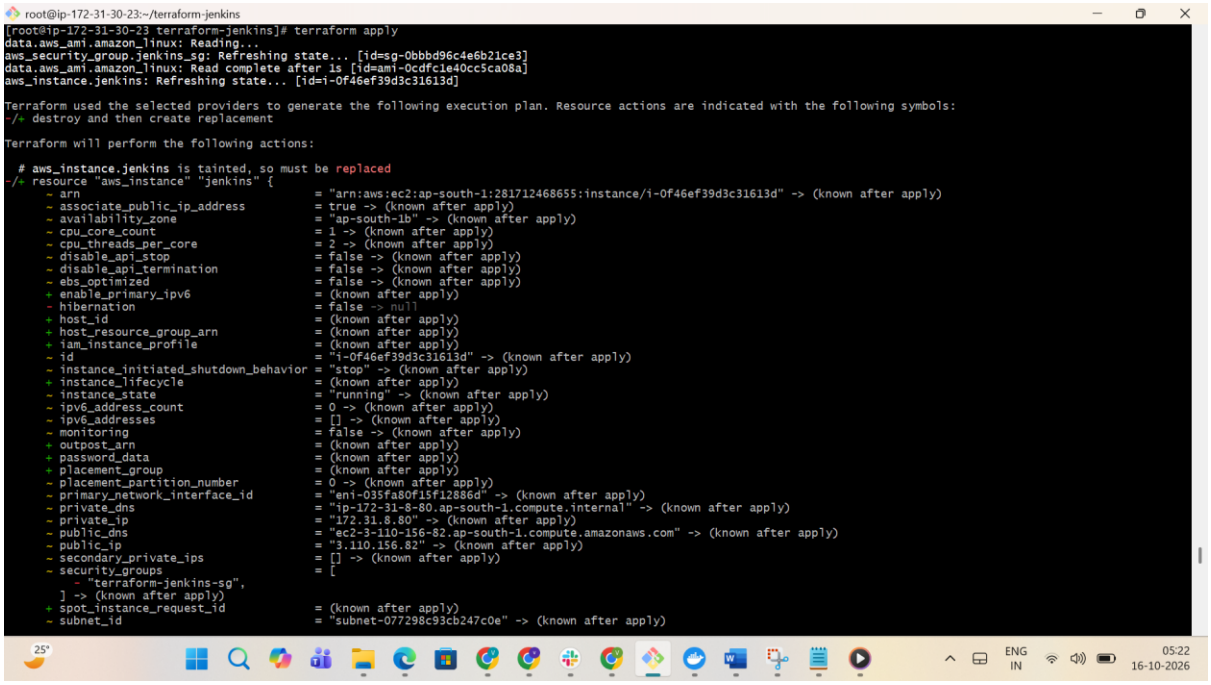
18. Opened Jenkins in a browser using the instance's public IP and configured Jenkins through the setup wizard.
19. Installed the required Jenkins plugins and configured the administrator account.
20. Verified that the Jenkins dashboard was accessible.

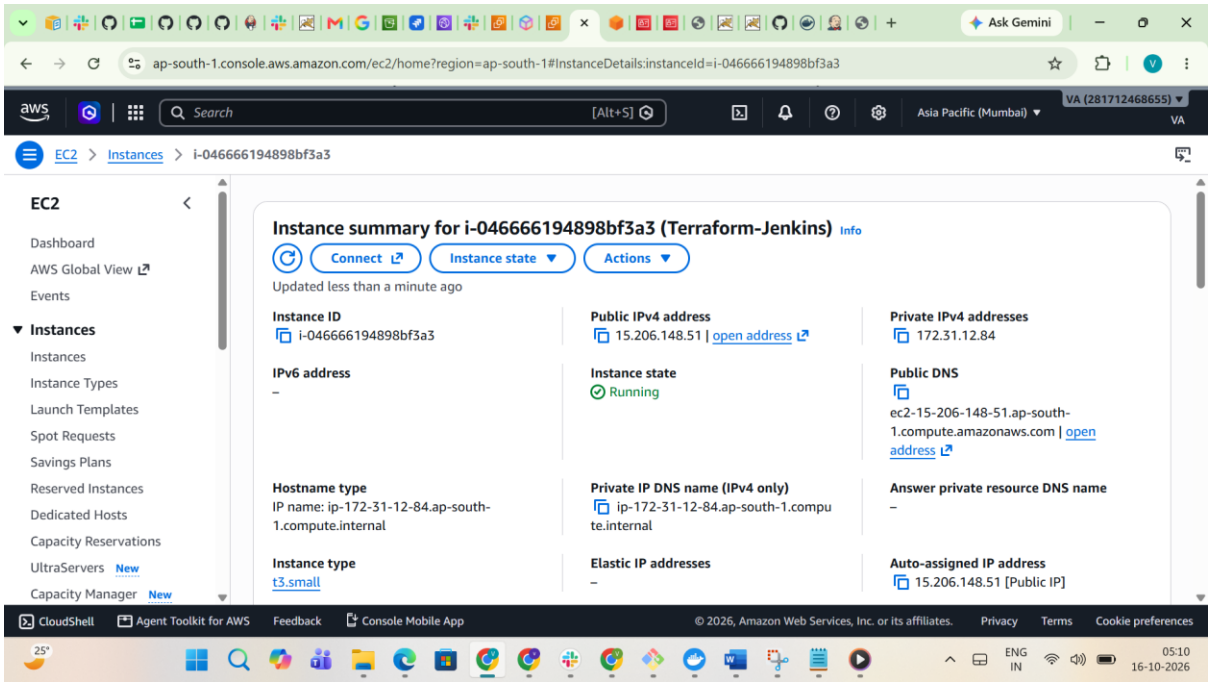

root@ip-172-31-30-23:~/terraform-jenkins
[root@ip-172-31-30-23 terraform-jenkins]# terraform init
-bash: terraform: command not found
[root@ip-172-31-30-23 terraform-jenkins]# ^C
[root@ip-172-31-30-23 terraform-jenkins]# dnf install -y dnf-plugins-core
Last metadata expiration check: 1 day, 13:14:09 ago on Wed Oct 14 10:02:03 2026.
Package dnf-plugins-core-4.3.0-13.amzn2023.0.7.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-30-23 terraform-jenkins]# dnf config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
Adding repo from: https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
[root@ip-172-31-30-23 terraform-jenkins]# dnf install -y terraform
Hashicorp Stable - x86_64 21 MB/s | 1.9 MB 00:00
Dependencies resolved.
Package Architecture Version Repository Size
Installing:
terraform x86_64 1.16.6-1 hashicorp 35 M
Transaction Summary
Install 1 Package
Total download size: 35 M
Installed size: 115 M
Downloading Packages:
terraform-1.16.6-1.x86_64.rpm 86 MB/s | 35 MB 00:00
Total 84 MB/s | 35 MB 00:00
Hashicorp Stable - x86_64 324 kB/s | 1.7 kB 00:00
Importing GPG key 0xCA026560:
Userid : "HashiCorp Security (HashiCorp Package Signing) <security+packaging@hashicorp.com>"
Fingerprint: D55C 0D1A C78A 8D81 26CB 631C FC9C A96A CA02 6560
From : https://rpm.releases.hashicorp.com/gpg
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing : 1/1
Installing : terraform-1.16.6-1.x86_64 1/1
Verifying : terraform-1.16.6-1.x86_64 1/1
WARNING:
A newer release of "Amazon Linux" is available.
ENG IN 05:19 16-10-2026

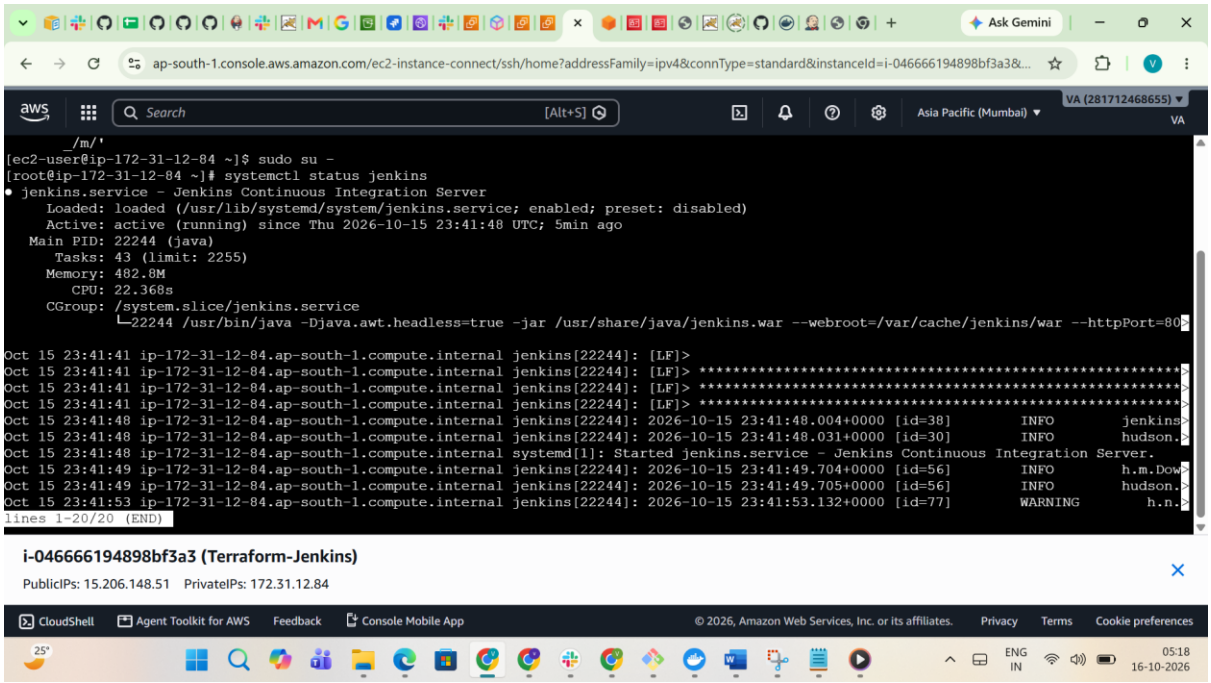

root@ip-172-31-30-23:~/terraform-jenkins
[root@ip-172-31-30-23 terraform-jenkins]# cat main.tf
terraform {
required_providers {
aws = {
source = "hashicorp/aws"
version = "~> 5.0"
}
}
}
provider "aws" {
region = "ap-south-1"
}
data "aws_ami" "amazon_linux" {
most_recent = true
owners = ["137112412989"]
filter {
name = "name"
values = ["al2023-ami-2023.*-x86_64"]
}
filter {
name = "architecture"
values = ["x86_64"]
}
}
resource "aws_security_group" "jenkins_sg" {
name = "terraform-jenkins-sg"
description = "Access for Jenkins lab"
ingress {
description = "SSH"
from_port = 22
to_port = 22
protocol = "tcp"
cidr_blocks = ["54.209.214.243/32"]
}
ingress {
description = "Jenkins web interface"
from_port = 8080
to_port = 8080
protocol = "tcp"
cidr_blocks = ["54.209.214.243/32"]
}
ENG IN 05:21 16-10-2026

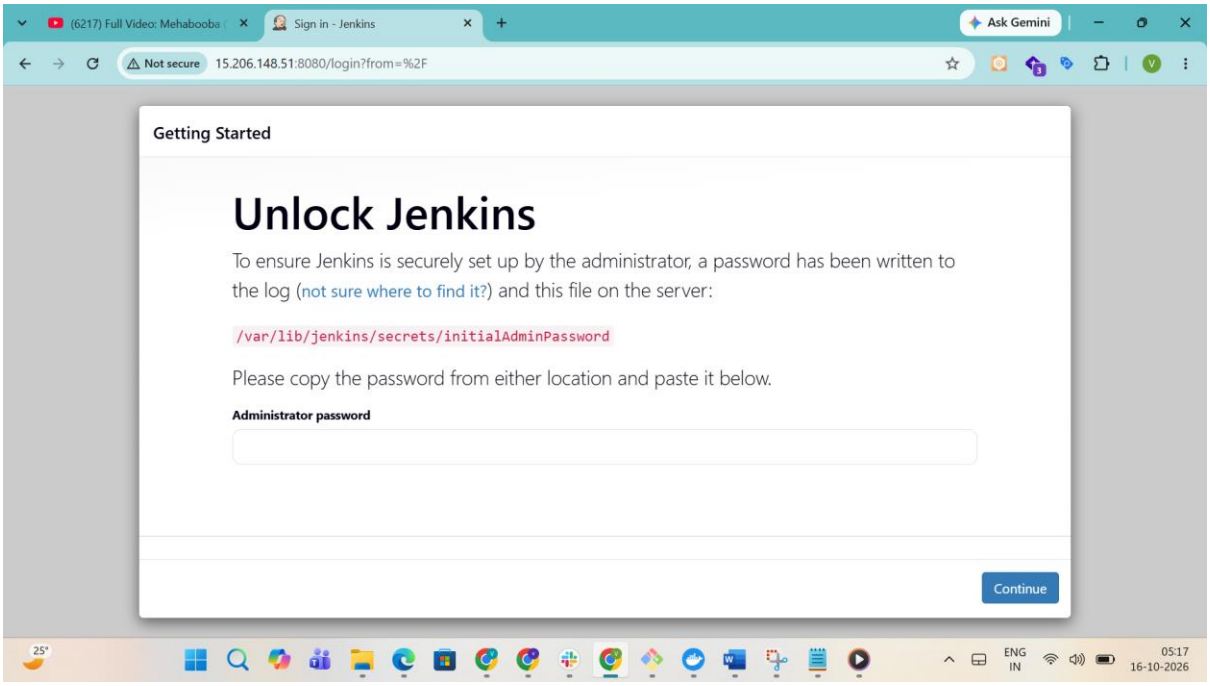

Installed:
terraform-1.16.6-1.x86_64
Complete!
[root@ip-172-31-30-23 terraform-jenkins]# terraform -version
Terraform v1.16.6
on linux_amd64
[root@ip-172-31-30-23 terraform-jenkins]# terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
Terraform has been successfully initialized!
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@ip-172-31-30-23 terraform-jenkins]# terraform validate


root@ip-172-31-30-23:~/terraform-jenkins
terraform plan
data.aws_ami.amazon_linux: Reading...
aws_security_group.jenkins_sg: Refreshing state... [id=sg-0bbbd96c4e6b21ce3]
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-0cdfc1e40cc5ca08a]
aws_instance.jenkins: Refreshing state... [id=i-0f46ef39d3c31613d]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement
Terraform will perform the following actions:
aws_instance.jenkins is tainted, so must be replaced
-/+ resource "aws_instance" "jenkins" {
~ arn = "arn:aws:ec2:ap-south-1:281712468655:instance/i-0f46ef39d3c31613d" -> (known after apply)
~ associate_public_ip_address = true -> (known after apply)
~ availability_zone = "ap-south-1b" -> (known after apply)
~ cpu_core_count = 1 -> (known after apply)
~ cpu_threads_per_core = 2 -> (known after apply)
~ disable_api_stop = false -> (known after apply)
~ disable_api_termination = false -> (known after apply)
~ ebs_optimized = false -> (known after apply)
+ enable_primary_ipv6 = (known after apply)
- hibernation = false -> null
+ host_id = (known after apply)
+ host_resource_group_arn = (known after apply)
+ iam_instance_profile = (known after apply)
~ id = "i-0f46ef39d3c31613d" -> (known after apply)
~ instance_initiated_shutdown_behavior = "stop" -> (known after apply)
+ instance_lifecycle = (known after apply)
~ instance_state = "running" -> (known after apply)
~ ipv6_address_count = 0 -> (known after apply)
~ ipv6_addresses = [] -> (known after apply)
~ monitoring = false -> (known after apply)
+ outpost_arn = (known after apply)
+ password_data = (known after apply)
+ placement_group = (known after apply)
~ placement_partition_number = 0 -> (known after apply)
~ primary_network_interface_id = "eni-035fa80f15f12886d" -> (known after apply)
~ private_dns = "ip-172-31-8-80.ap-south-1.compute.internal" -> (known after apply)
~ private_ip = "172.31.8.80" -> (known after apply)
~ public_dns = "ec2-3-110-156-82.ap-south-1.compute.amazonaws.com" -> (known after apply)
~ public_ip = "3.110.156.82" -> (known after apply)
~ secondary_private_ips = [] -> (known after apply)
~ security_groups = [
- "terraform-jenkins-sg",
] -> (known after apply)
+ spot_instance_request_id = (known after apply)
25°
ENG
IN
05:23
16-10-2026


root@ip-172-31-30-23:~/terraform-jenkins
[root@ip-172-31-30-23 terraform-jenkins]# terraform apply
data.aws_ami.amazon_linux: Reading...
aws_security_group.jenkins_sg: Refreshing state... [id=sg-0bbbd96c4e6b21ce3]
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-0cdfc1e40cc5ca08a]
aws_instance.jenkins: Refreshing state... [id=i-0f46ef39d3c31613d]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement
Terraform will perform the following actions:
aws_instance.jenkins is tainted, so must be replaced
-/+ resource "aws_instance" "jenkins" {
~ arn = "arn:aws:ec2:ap-south-1:281712468655:instance/i-0f46ef39d3c31613d" -> (known after apply)
~ associate_public_ip_address = true -> (known after apply)
~ availability_zone = "ap-south-1b" -> (known after apply)
~ cpu_core_count = 1 -> (known after apply)
~ cpu_threads_per_core = 2 -> (known after apply)
~ disable_api_stop = false -> (known after apply)
~ disable_api_termination = false -> (known after apply)
~ ebs_optimized = false -> (known after apply)
+ enable_primary_ipv6 = (known after apply)
- hibernation = false -> null
+ host_id = (known after apply)
+ host_resource_group_arn = (known after apply)
+ iam_instance_profile = (known after apply)
~ id = "i-0f46ef39d3c31613d" -> (known after apply)
~ instance_initiated_shutdown_behavior = "stop" -> (known after apply)
+ instance_lifecycle = (known after apply)
~ instance_state = "running" -> (known after apply)
~ ipv6_address_count = 0 -> (known after apply)
~ ipv6_addresses = [] -> (known after apply)
~ monitoring = false -> (known after apply)
+ outpost_arn = (known after apply)
+ password_data = (known after apply)
+ placement_group = (known after apply)
~ placement_partition_number = 0 -> (known after apply)
~ primary_network_interface_id = "eni-035fa80f15f12886d" -> (known after apply)
~ private_dns = "ip-172-31-8-80.ap-south-1.compute.internal" -> (known after apply)
~ private_ip = "172.31.8.80" -> (known after apply)
~ public_dns = "ec2-3-110-156-82.ap-south-1.compute.amazonaws.com" -> (known after apply)
~ public_ip = "3.110.156.82" -> (known after apply)
~ secondary_private_ips = [] -> (known after apply)
~ security_groups = [
- "terraform-jenkins-sg",
] -> (known after apply)
+ spot_instance_request_id = (known after apply)
~ subnet_id = "subnet-077298c93cb247c0e" -> (known after apply)
25°
ENG
IN
05:22
16-10-2026


ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#InstanceDetails:instanceId=i-046666194898bf3a3
Search
[Alt+S]
Asia Pacific (Mumbai)
VA (281712468655)
VA
EC2 > Instances > i-046666194898bf3a3
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Instance summary for i-046666194898bf3a3 (Terraform-Jenkins) Info
Connect
Instance state
Actions
Updated less than a minute ago
Instance ID
i-046666194898bf3a3
Public IPv4 address
15.206.148.51 | open address
Private IPv4 addresses
172.31.12.84
IPv6 address
–
Instance state
Running
Public DNS
ec2-15-206-148-51.ap-south-1.compute.amazonaws.com | open address
Hostname type
IP name: ip-172-31-12-84.ap-south-1.compute.internal
Private IP DNS name (IPv4 only)
ip-172-31-12-84.ap-south-1.compute.internal
Answer private resource DNS name
–
Instance type
t3.small
Elastic IP addresses
–
Auto-assigned IP address
15.206.148.51 [Public IP]
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
05:10
16-10-2026


ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-046666194898bf3a3&...
Search
[Alt+S]
Asia Pacific (Mumbai)
VA (281712468655)
VA
_/m/'
[ec2-user@ip-172-31-12-84 ~]$ sudo su -
[root@ip-172-31-12-84 ~]# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
Active: active (running) since Thu 2026-10-15 23:41:48 UTC; 5min ago
Main PID: 22244 (java)
Tasks: 43 (limit: 2255)
Memory: 482.8M
CPU: 22.368s
CGroup: /system.slice/jenkins.service
└22244 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=80
Oct 15 23:41:41 ip-172-31-12-84.ap-south-1.compute.internal jenkins[22244]: [LF]>
Oct 15 23:41:41 ip-172-31-12-84.ap-south-1.compute.internal jenkins[22244]: [LF]> **********
Oct 15 23:41:41 ip-172-31-12-84.ap-south-1.compute.internal jenkins[22244]: [LF]> **********
Oct 15 23:41:41 ip-172-31-12-84.ap-south-1.compute.internal jenkins[22244]: [LF]> **********
Oct 15 23:41:48 ip-172-31-12-84.ap-south-1.compute.internal jenkins[22244]: 2026-10-15 23:41:48.004+0000 [id=38] INFO jenkins
Oct 15 23:41:48 ip-172-31-12-84.ap-south-1.compute.internal jenkins[22244]: 2026-10-15 23:41:48.031+0000 [id=30] INFO hudson.
Oct 15 23:41:48 ip-172-31-12-84.ap-south-1.compute.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Oct 15 23:41:49 ip-172-31-12-84.ap-south-1.compute.internal jenkins[22244]: 2026-10-15 23:41:49.704+0000 [id=56] INFO h.m.Dow
Oct 15 23:41:49 ip-172-31-12-84.ap-south-1.compute.internal jenkins[22244]: 2026-10-15 23:41:49.705+0000 [id=56] INFO hudson.
Oct 15 23:41:53 ip-172-31-12-84.ap-south-1.compute.internal jenkins[22244]: 2026-10-15 23:41:53.132+0000 [id=77] WARNING h.n.
lines 1-20/20 (END)
i-046666194898bf3a3 (Terraform-Jenkins)
PublicIPs: 15.206.148.51 PrivateIPs: 172.31.12.84
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
05:18
16-10-2026



Question 5: Create a Jenkins Job to Build and Push a Docker Image to Docker Hub

Source Repository: https://github.com/betawins/Python-app.git

Implementation Steps

1. Opened the Jenkins dashboard.
2. Installed the required plugins, including Git, Pipeline, and Docker-related tools as needed.
3. Verified that Git, Docker CLI, and the required Python runtime or build environment were available to the Jenkins execution environment.
4. Created a Docker Hub repository for the Python application.
5. Opened **Manage Jenkins → Credentials** and added Docker Hub credentials.
6. Assigned a credential ID such as dockerhub-creds.
7. Created a new Jenkins Pipeline job.
8. Configured the pipeline to retrieve the source code from:

   https://github.com/betawins/Python-app.git
9. Added a Dockerfile to the application repository if one was not already provided.
10. Configured the pipeline to clone the repository, build the Docker image, authenticate with Docker Hub, and push the image.
11. Used Jenkins credentials binding instead of hardcoding Docker Hub credentials in the pipeline.
12. Used the following declarative pipeline as an example, adjusting the Dockerfile location and application ports to match the repository:

    pipeline {

```
agent any

environment {
  IMAGE_NAME = 'DOCKERHUB_USERNAME/python-app'
}

stages {
  stage('Checkout') {
    steps {
      git branch: 'main',
        url: 'https://github.com/betawins/Python-app.git'
    }
  }

  stage('Build Docker Image') {
    steps {
      sh '''
        docker build -t ${IMAGE_NAME}:${BUILD_NUMBER} .
        docker tag ${IMAGE_NAME}:${BUILD_NUMBER} ${IMAGE_NAME}:latest
      '''
    }
  }

  stage('Push to Docker Hub') {
    steps {
      withCredentials([
        usernamePassword(
          credentialsId: 'dockerhub-creds',
          usernameVariable: 'DOCKER_USER',
          passwordVariable: 'DOCKER_PASS'
        )
```

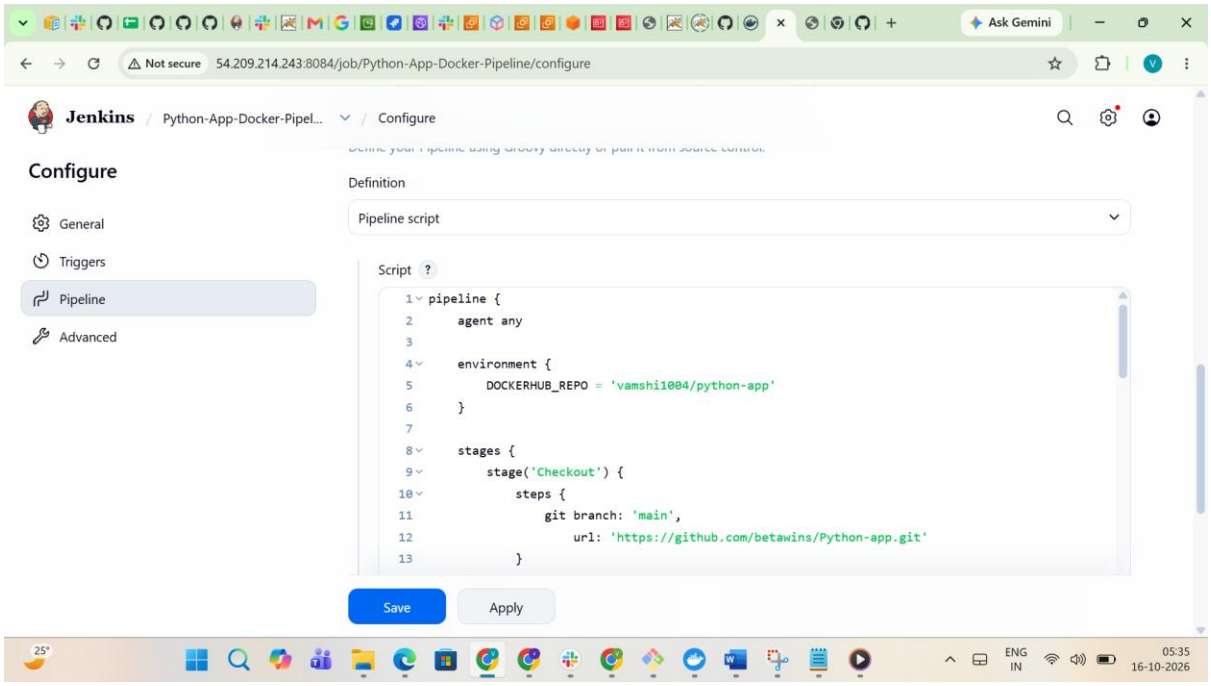
```
        ]) {
          sh '''
            set +x
            echo "$DOCKER_PASS" |
              docker login -u "$DOCKER_USER" --password-stdin

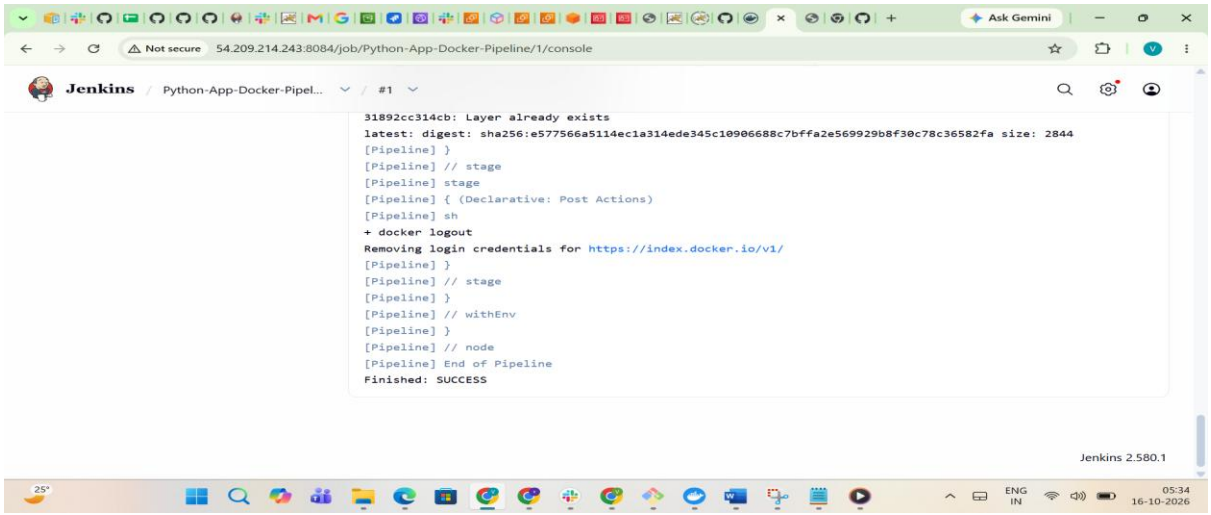
            docker push ${IMAGE_NAME}:${BUILD_NUMBER}
            docker push ${IMAGE_NAME}:latest

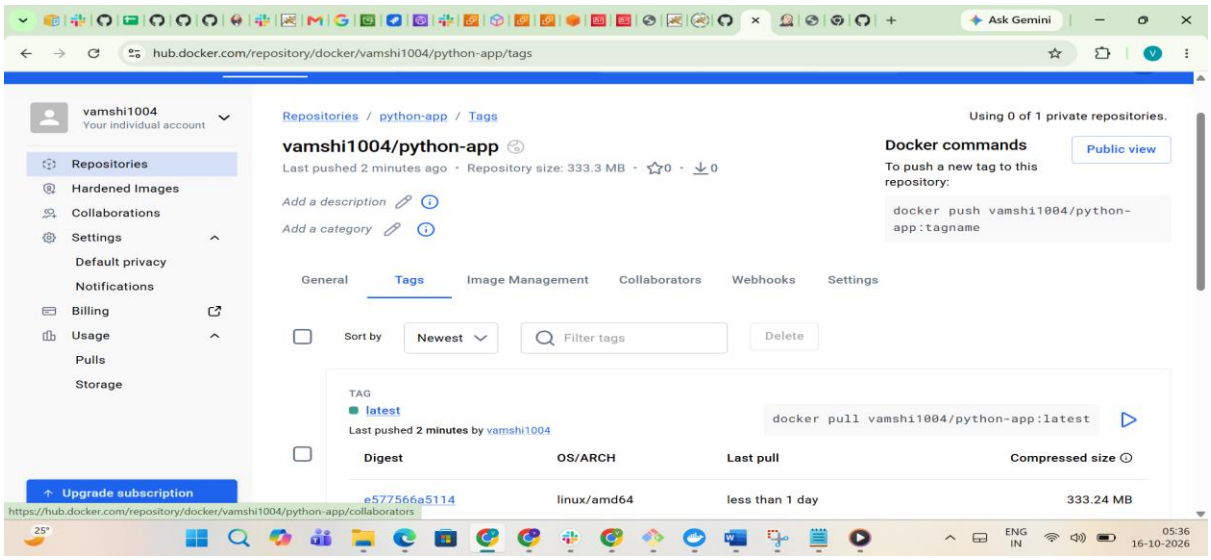
            docker logout
          '''
        }
      }
    }
  }
}
```

Replace DOCKERHUB_USERNAME with the actual Docker Hub username. If the repository uses a branch other than main, update the branch name.

13. Saved the Jenkins pipeline configuration.
14. Selected **Build Now** to execute the job.
15. Opened **Console Output** and verified that the source code was checked out, the image was built, and the image was pushed successfully.
16. Opened Docker Hub and confirmed that both the build-number tag and latest tag were available.







Question 6: Create a Dockerfile for Frontend Source Code, Build, Run and Push the Image

Source Repository: https://github.com/betawins/docker-tasks.git

Implementation Steps

1. Connected to the EC2 instance.
2. Cloned the source repository:

   git clone https://github.com/betawins/docker-tasks.git
3. Navigated to the frontend source-code directory.
4. Reviewed the application files, including HTML, CSS, JavaScript, and any required assets.
5. Created a Dockerfile using Nginx to serve the static frontend files:

   FROM nginx:stable-alpine

   COPY . /usr/share/nginx/html/

   EXPOSE 80

   CMD ["nginx", "-g", "daemon off;"]
6. Built the frontend image:

   docker build -t frontend-app:v1 .
7. Created and started the frontend container:

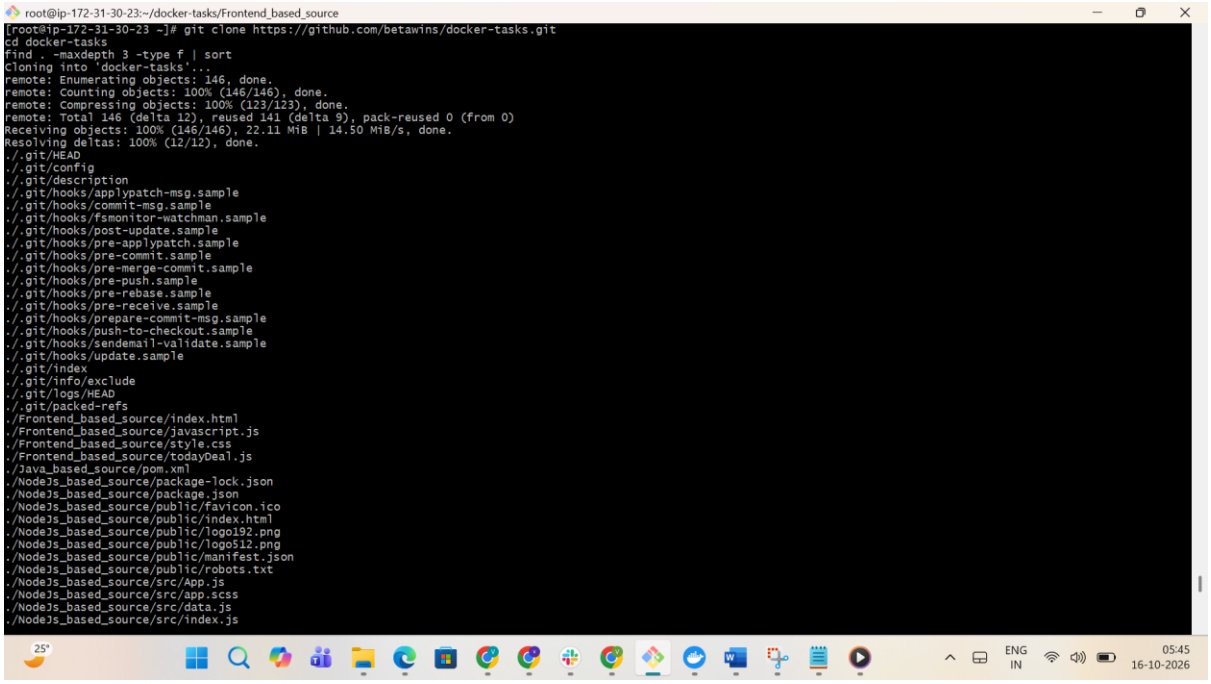
   docker run -d --name frontend-app -p 8086:80 frontend-app:v1
8. Verified that the container was running:

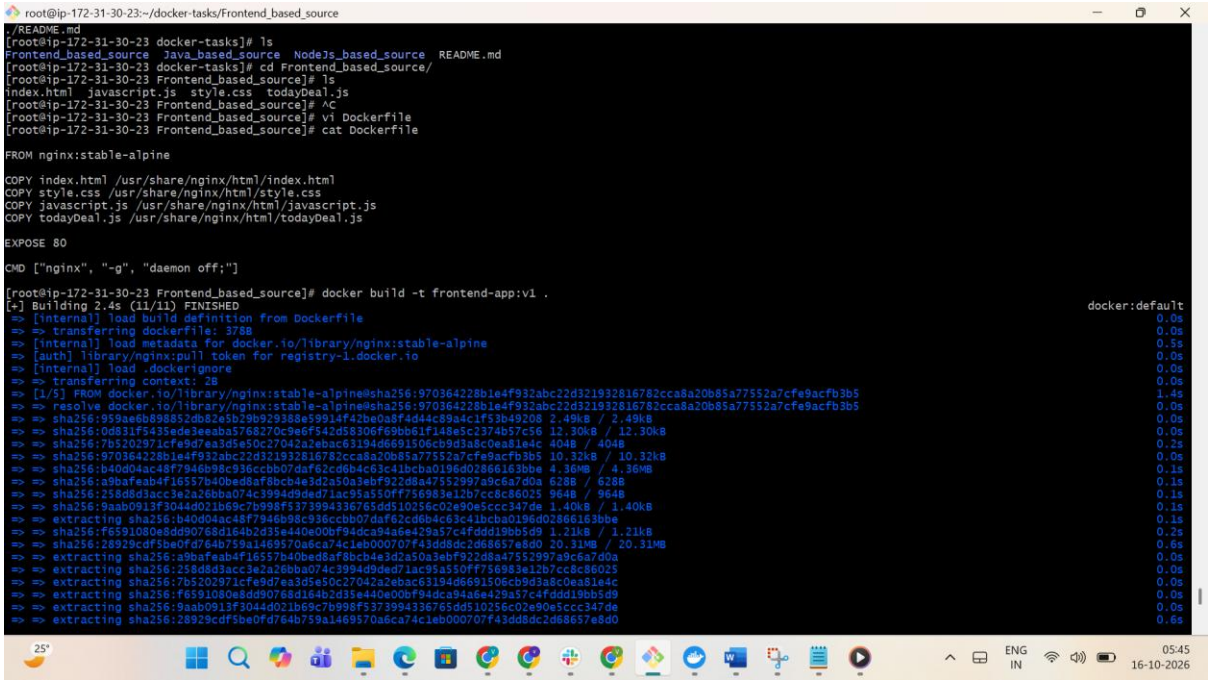
   docker ps
9. Tested the frontend using http://EC2_PUBLIC_IP:8086, after allowing TCP port 8085 in the EC2 security group.
10. Logged in to Docker Hub and tagged the image:

    docker login

    docker tag frontend-app:v1 DOCKERHUB_USERNAME/frontend-app:v1
11. Pushed the image:

    docker push DOCKERHUB_USERNAME/frontend-app:v1
12. Verified the image in Docker Hub.


root@ip-172-31-30-23:~/docker-tasks/Frontend_based_source
[root@ip-172-31-30-23 ~]# git clone https://github.com/betawins/docker-tasks.git
cd docker-tasks
find . -maxdepth 3 -type f | sort
Cloning into 'docker-tasks'...
remote: Enumerating objects: 146, done.
remote: Counting objects: 100% (146/146), done.
remote: Compressing objects: 100% (123/123), done.
remote: Total 146 (delta 12), reused 141 (delta 9), pack-reused 0 (from 0)
Receiving objects: 100% (146/146), 22.11 MiB | 14.50 MiB/s, done.
Resolving deltas: 100% (12/12), done.
./.git/HEAD
./.git/config
./.git/description
./.git/hooks/applypatch-msg.sample
./.git/hooks/commit-msg.sample
./.git/hooks/fsmonitor-watchman.sample
./.git/hooks/post-update.sample
./.git/hooks/pre-applypatch.sample
./.git/hooks/pre-commit.sample
./.git/hooks/pre-merge-commit.sample
./.git/hooks/pre-push.sample
./.git/hooks/pre-rebase.sample
./.git/hooks/pre-receive.sample
./.git/hooks/prepare-commit-msg.sample
./.git/hooks/push-to-checkout.sample
./.git/hooks/sendemail-validate.sample
./.git/hooks/update.sample
./.git/index
./.git/info/exclude
./.git/logs/HEAD
./.git/packed-refs
./Frontend_based_source/index.html
./Frontend_based_source/javascript.js
./Frontend_based_source/style.css
./Frontend_based_source/todayDeal.js
./Java_based_source/pom.xml
./NodeJs_based_source/package-lock.json
./NodeJs_based_source/package.json
./NodeJs_based_source/public/favicon.ico
./NodeJs_based_source/public/index.html
./NodeJs_based_source/public/logo192.png
./NodeJs_based_source/public/logo512.png
./NodeJs_based_source/public/manifest.json
./NodeJs_based_source/public/robots.txt
./NodeJs_based_source/src/App.js
./NodeJs_based_source/src/app.scss
./NodeJs_based_source/src/data.js
./NodeJs_based_source/src/index.js
25°
ENG
IN
05:45
16-10-2026


root@ip-172-31-30-23:~/docker-tasks/Frontend_based_source
./README.md
[root@ip-172-31-30-23 docker-tasks]# ls
Frontend_based_source Java_based_source NodeJs_based_source README.md
[root@ip-172-31-30-23 docker-tasks]# cd Frontend_based_source/
[root@ip-172-31-30-23 Frontend_based_source]# ls
index.html javascript.js style.css todayDeal.js
[root@ip-172-31-30-23 Frontend_based_source]# ^C
[root@ip-172-31-30-23 Frontend_based_source]# vi Dockerfile
[root@ip-172-31-30-23 Frontend_based_source]# cat Dockerfile

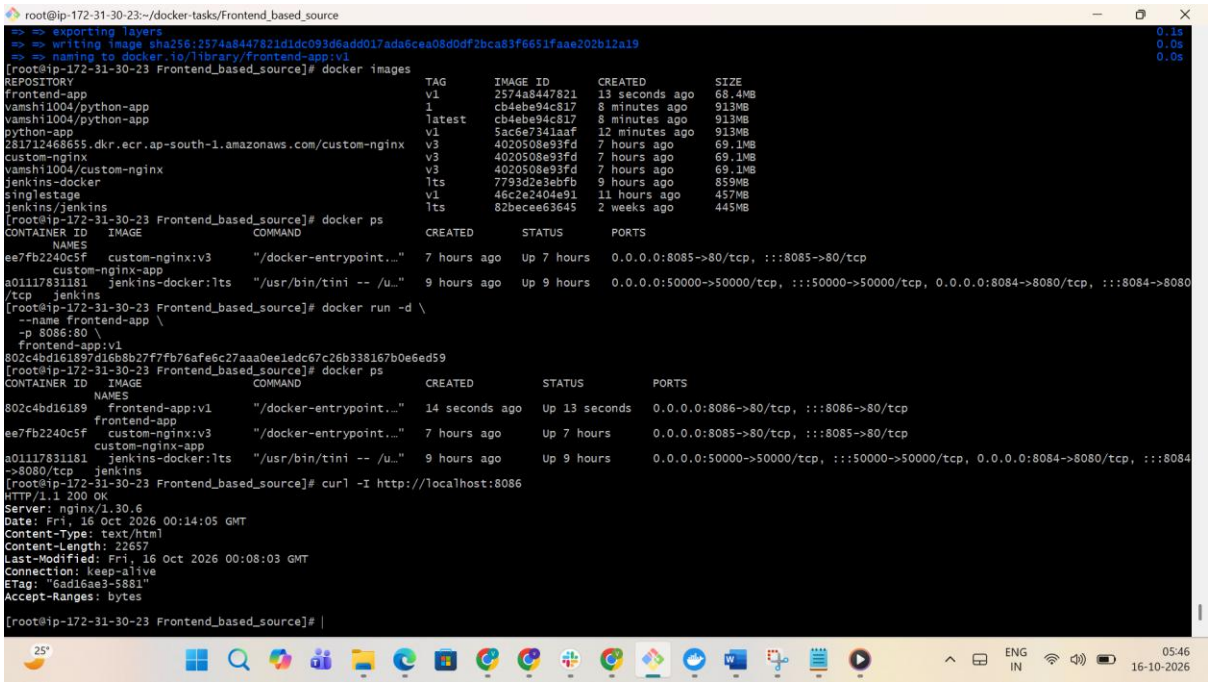
FROM nginx:stable-alpine

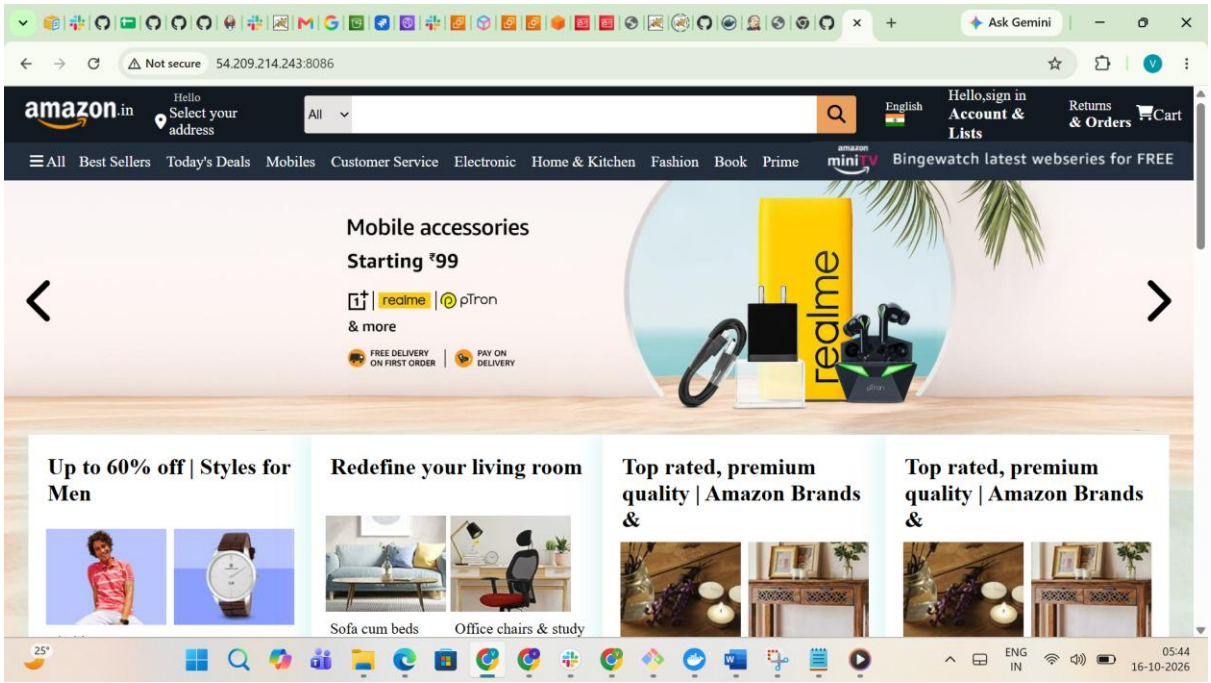
COPY index.html /usr/share/nginx/html/index.html
COPY style.css /usr/share/nginx/html/style.css
COPY javascript.js /usr/share/nginx/html/javascript.js
COPY todayDeal.js /usr/share/nginx/html/todayDeal.js

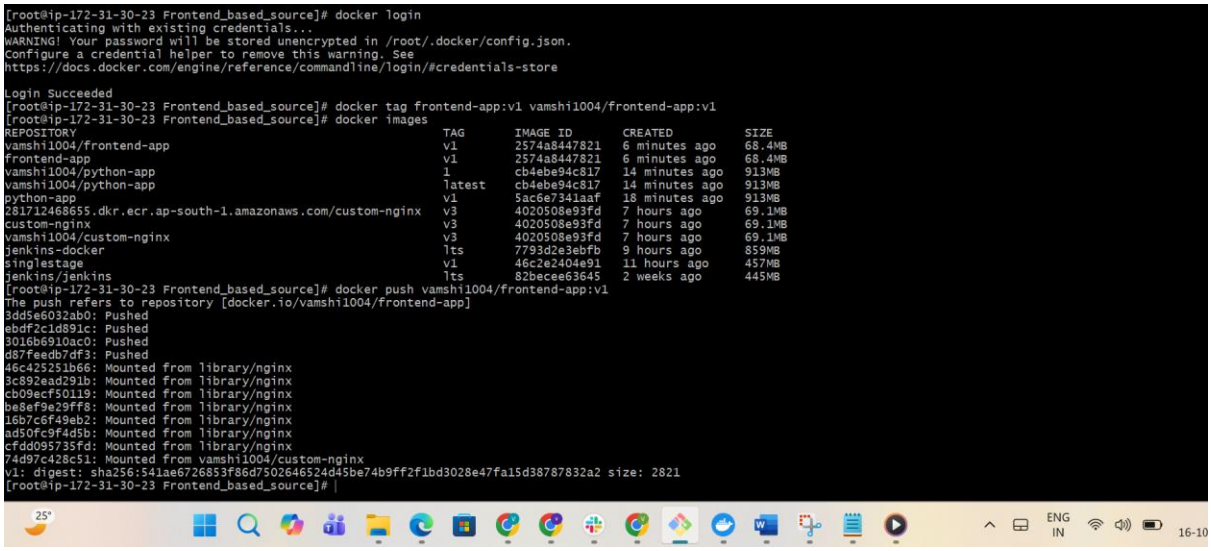
EXPOSE 80

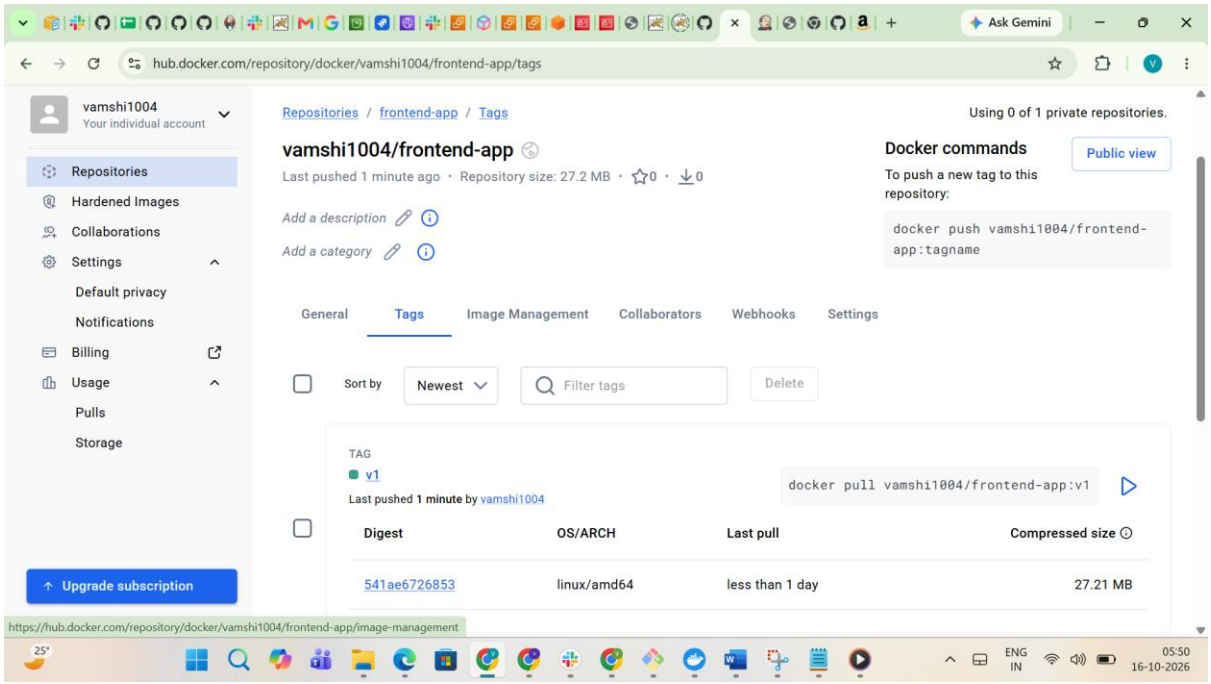
CMD ["nginx", "-g", "daemon off;"]

[root@ip-172-31-30-23 Frontend_based_source]# docker build -t frontend-app:v1 .
[+] Building 2.4s (11/11) FINISHED
docker:default
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 378B
=> [internal] load metadata for docker.io/library/nginx:stable-alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/nginx:stable-alpine@sha256:970364228b1e4f932abc22d3219328l6782cca8a20b85a77552a7cfe9acfb3b5
=> => resolve docker.io/library/nginx:stable-alpine@sha256:970364228b1e4f932abc22d3219328l6782cca8a20b85a77552a7cfe9acfb3b5
=> => sha256:959ae6b898852db82e5b29b929388e59914f42be0a8f4d44c89a4c1f53b49208 2.49kB / 2.49kB
=> => sha256:0d831f5435ede3eeaba5768270c9e6f542d58306f69bb61f148e5c2374b57c56 12.30kB / 12.30kB
=> => sha256:7b5202971cfe9d7ea3d5e50c27042a2ebac63194d6691506cb9d3a8c0ea81e4c 404B / 404B
=> => sha256:970364228b1e4f932abc22d321932816782cca8a20b85a77552a7cfe9acfb3b5 10.32kB / 10.32kB
=> => sha256:b40d04ac48f7946b98c936ccbb07daf62cd6b4c63c41bcba0196d02866163bbe 4.36MB / 4.36MB
=> => sha256:a9bafeab4f16557b40bed8af8bcb4e3d2a50a3ebf922d8a47552997a9c6a7d0a 628B / 628B
=> => sha256:258d8d3acc3e2a26bba074c3994d9ded71ac95a550ff756983e12b7cc8c86025 964B / 964B
=> => sha256:9aab0913f3044d021b69c7b998f537399433676 5dd510256c02e90e5ccc347de 1.40kB / 1.40kB
=> => extracting sha256:b40d04ac48f7946b98c936ccbb07daf62cd6b4c63c41bcba0196d02866163bbe
=> => sha256:f6591080e8dd90768d164b2d35e440e00bf94dca94a6e429a57c4fddd19bb5d9 1.21kB / 1.21kB
=> => sha256:28929cdf5be0fd764b759a1469570a6ca74c1eb000707f43dd8dc2d68657e8d0 20.31MB / 20.31MB
=> => extracting sha256:a9bafeab4f16557b40bed8af8bcb4e3d2a50a3ebf922d8a47552997a9c6a7d0a
=> => extracting sha256:258d8d3acc3e2a26bba074c3994d9ded71ac95a550ff756983e12b7cc8c86025
=> => extracting sha256:7b5202971cfe9d7ea3d5e50c27042a2ebac63194d6691506cb9d3a8c0ea81e4c
=> => extracting sha256:f6591080e8dd90768d164b2d35e440e00bf94dca94a6e429a57c4fddd19bb5d9
=> => extracting sha256:9aab0913f3044d021b69c7b998f537399433676 5dd510256c02e90e5ccc347de
=> => extracting sha256:28929cdf5be0fd764b759a1469570a6ca74c1eb000707f43dd8dc2d68657e8d0
25°
ENG
IN
05:45
16-10-2026


root@ip-172-31-30-23:~/docker-tasks/Frontend_based_source
=> => exporting layers 0.1s
=> => writing image sha256:2574a8447821d1dc093d6add017ada6cea08d0df2bca83f6651faae202b12a19 0.0s
=> => naming to docker.io/library/frontend-app:v1 0.0s
[root@ip-172-31-30-23 Frontend_based_source]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
frontend-app v1 2574a8447821 13 seconds ago 68.4MB
vamshi1004/python-app 1 cb4ebe94c817 8 minutes ago 913MB
vamshi1004/python-app latest cb4ebe94c817 8 minutes ago 913MB
python-app v1 5ac6e7341aaf 12 minutes ago 913MB
281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx v3 4020508e93fd 7 hours ago 69.1MB
custom-nginx v3 4020508e93fd 7 hours ago 69.1MB
vamshi1004/custom-nginx v3 4020508e93fd 7 hours ago 69.1MB
jenkins-docker lts 7793d2e3ebfb 9 hours ago 859MB
singlestage v1 46c2e2404e91 11 hours ago 457MB
jenkins/jenkins lts 82becee63645 2 weeks ago 445MB
[root@ip-172-31-30-23 Frontend_based_source]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
ee7fb2240c5f custom-nginx:v3 "/docker-entrypoint..." 7 hours ago Up 7 hours 0.0.0.0:8085->80/tcp, :::8085->80/tcp custom-nginx-app
a01117831181 jenkins-docker:lts "/usr/bin/tini -- /u..." 9 hours ago Up 9 hours 0.0.0.0:50000->50000/tcp, :::50000->50000/tcp, 0.0.0.0:8084->8080/tcp, :::8084->8080/tcp jenkins
[root@ip-172-31-30-23 Frontend_based_source]# docker run -d \
--name frontend-app \
-p 8086:80 \
frontend-app:v1
802c4bd161897d16b8b27f7fb76afe6c27aaa0ee1edc67c26b338167b0e6ed59
[root@ip-172-31-30-23 Frontend_based_source]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
802c4bd16189 frontend-app:v1 "/docker-entrypoint..." 14 seconds ago Up 13 seconds 0.0.0.0:8086->80/tcp, :::8086->80/tcp frontend-app
ee7fb2240c5f custom-nginx:v3 "/docker-entrypoint..." 7 hours ago Up 7 hours 0.0.0.0:8085->80/tcp, :::8085->80/tcp custom-nginx-app
a01117831181 jenkins-docker:lts "/usr/bin/tini -- /u..." 9 hours ago Up 9 hours 0.0.0.0:50000->50000/tcp, :::50000->50000/tcp, 0.0.0.0:8084->8080/tcp, :::8084->8080/tcp jenkins
[root@ip-172-31-30-23 Frontend_based_source]# curl -I http://localhost:8086
HTTP/1.1 200 OK
Server: nginx/1.30.6
Date: Fri, 16 Oct 2026 00:14:05 GMT
Content-Type: text/html
Content-Length: 22657
Last-Modified: Fri, 16 Oct 2026 00:08:03 GMT
Connection: keep-alive
ETag: "6ad16ae3-5881"
Accept-Ranges: bytes
[root@ip-172-31-30-23 Frontend_based_source]#
25°
ENG IN
05:46 16-10-2026


Ask Gemini
Not secure 54.209.214.243:8086
amazon.in
Hello Select your address
All
English
Hello,sign in Account & Lists
Returns & Orders
Cart
All Best Sellers Today's Deals Mobiles Customer Service Electronic Home & Kitchen Fashion Book Prime
miniTV Bingewatch latest webseries for FREE
Mobile accessories
Starting ₹99
realme pTron
& more
FREE DELIVERY ON FIRST ORDER
PAY ON DELIVERY
realme
Up to 60% off | Styles for Men
Redefine your living room
Sofa cum beds
Office chairs & study
Top rated, premium quality | Amazon Brands &
Top rated, premium quality | Amazon Brands &
25°
ENG IN
05:44 16-10-2026





Question 7: Create a Dockerfile for Java-Based Source Code, Build, Run and Push the Image

Source Repository: https://github.com/betawins/docker-tasks.git

Implementation Steps

1. Cloned the source repository and navigated to the Java application directory.
2. Reviewed the project structure and identified the build tool, such as Maven or Gradle.
3. Verified that the project could be built and that the generated JAR or WAR file was available.
4. Created a Dockerfile appropriate for the application. For a Maven-based application that produces an executable JAR, an example is:

   FROM maven:3.9-eclipse-temurin-17 AS build

```
WORKDIR /app

COPY pom.xml .

COPY src ./src


RUN mvn clean package -DskipTests


FROM eclipse-temurin:17-jre


WORKDIR /app

COPY --from=build /app/target/*.jar app.jar


EXPOSE 8080


ENTRYPOINT ["java", "-jar", "app.jar"]
```

5. Adjusted the Java version, build configuration, artifact name, and application port according to the actual source code.
6. Built the image:

   ```
   docker build -t java-app:v1 .
   ```

7. Verified the image:

   ```
   docker images
   ```

8. Ran the Java application:

   ```
   docker run -d --name java-app -p 8087:8080 java-app:v1
   ```

9. Checked the container status and application logs:

   ```
   docker ps

   docker logs java-app
   ```

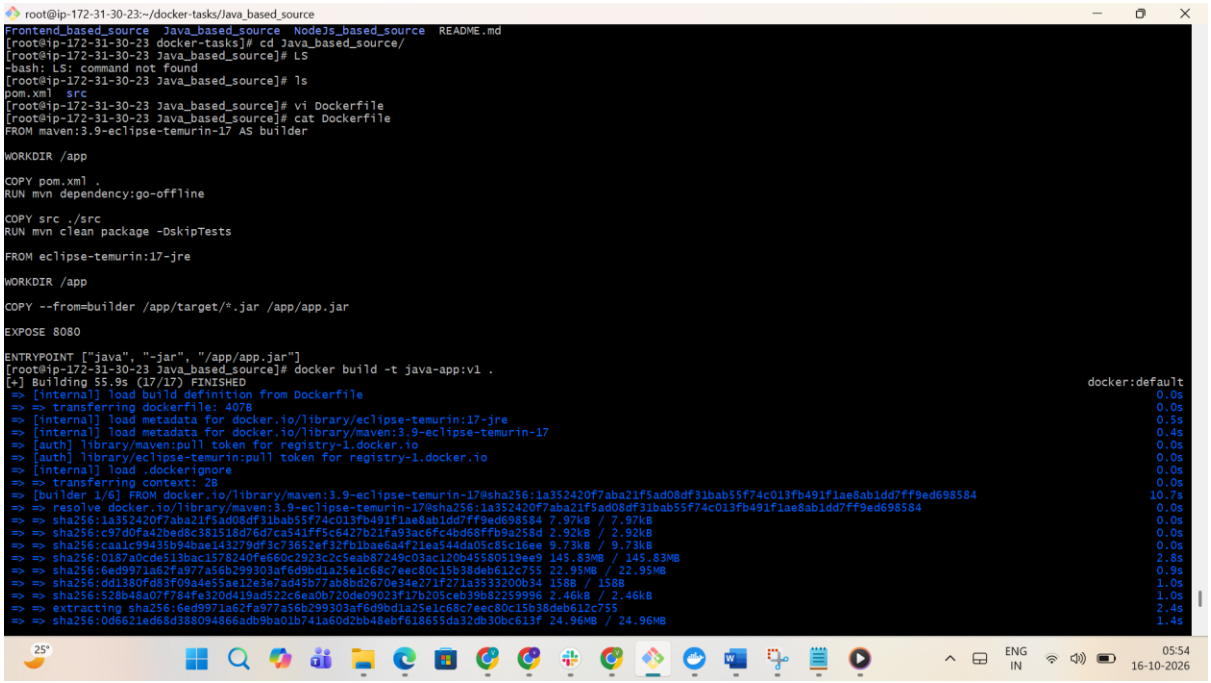
10. Tested the application using its appropriate HTTP endpoint.
11. Logged in to Docker Hub.
12. Tagged the image:

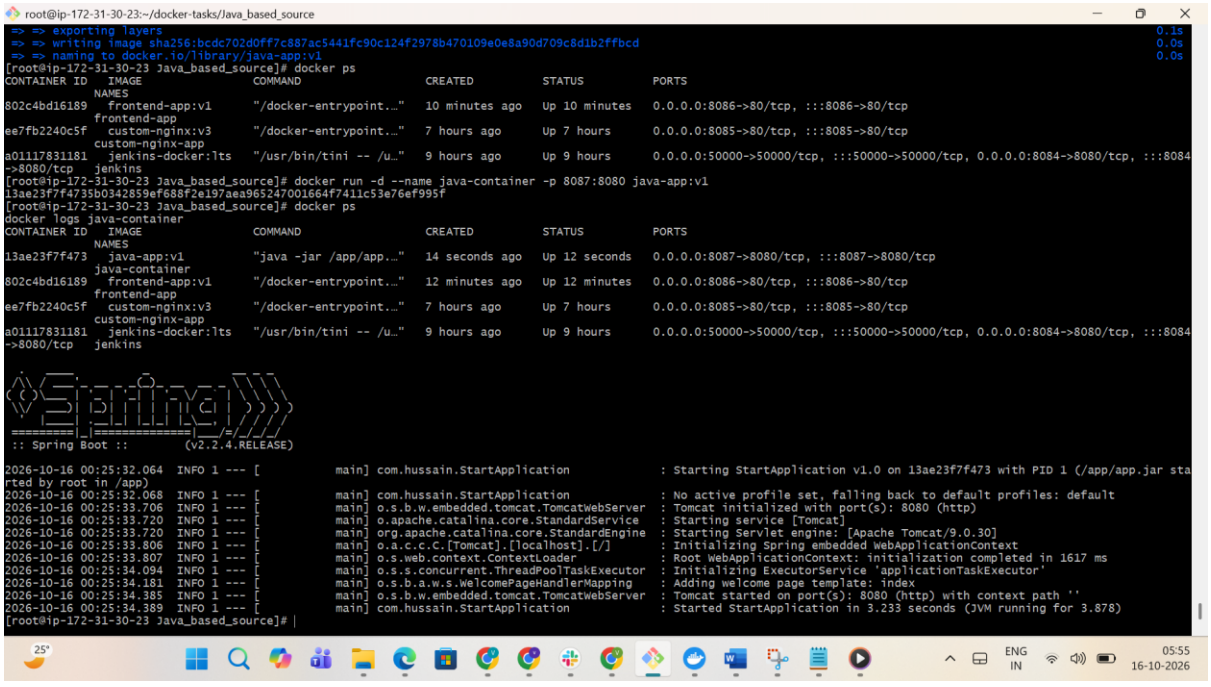
    ```
    docker tag java-app:v1 DOCKERHUB_USERNAME/java-app:v1
    ```

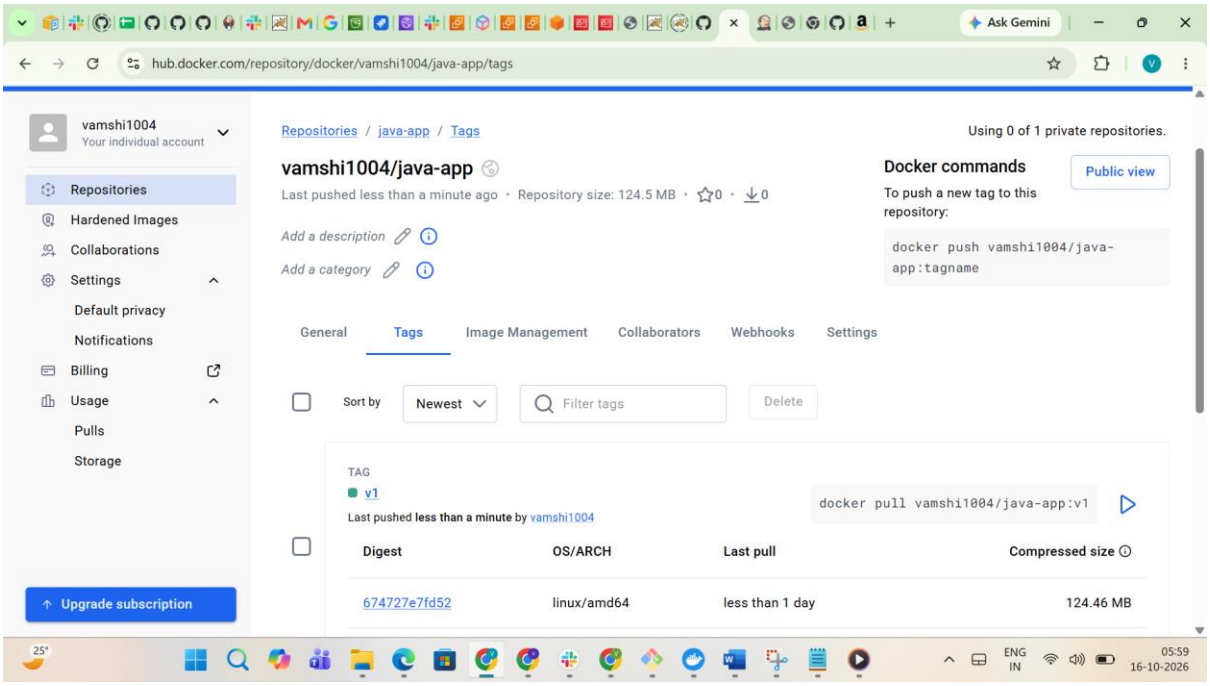
13. Pushed the image:

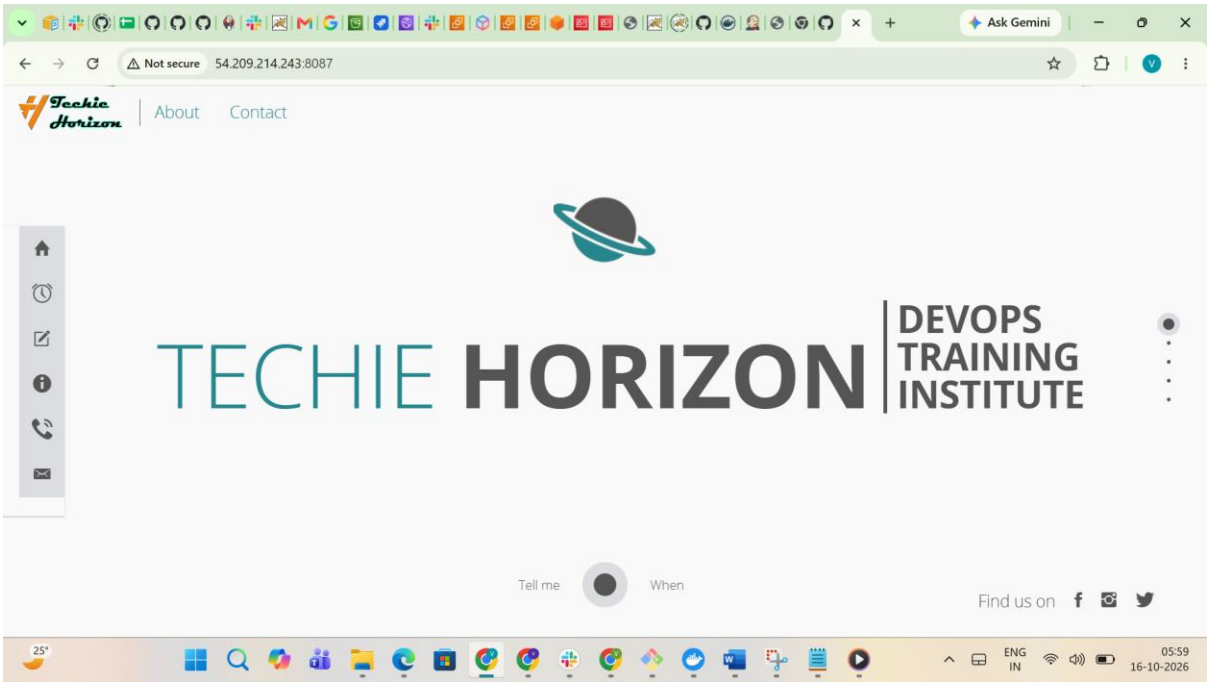
    ```
    docker push DOCKERHUB_USERNAME/java-app:v1
    ```

14. Verified the image and tag in Docker Hub.




root@ip-172-31-30-23:~/docker-tasks/Java_based_source
=> => exporting layers
=> => writing image sha256:bcdc702d0ff7c887ac5441fc90c124f2978b470109e0e8a90d709c8d1b2ffbcd
=> => naming to docker.io/library/java-app:v1
0.1s
0.0s
0.0s
[root@ip-172-31-30-23 Java_based_source]# docker ps
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS





Question 8: Create a Dockerfile for a Node.js Application, Build with Tag v1, Run and Push the Image

Source Repository: https://github.com/betawins/docker-tasks.git

Implementation Steps

1. Cloned the repository and navigated to the Node.js application directory.
2. Reviewed package.json, the lock file, and the application startup command.
3. Created a Dockerfile:

   FROM node:22-alpine

```
WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

4. Built the image with the required v1 tag:

   docker build -t frontend-app:v1 .

5. Verified that the image was created:

   docker images

6. Started the container:

   docker run -d --name node-app -p 8088:3000 frontend-app:v1

7. Checked the container status and application logs:

   docker ps

   docker logs node-app

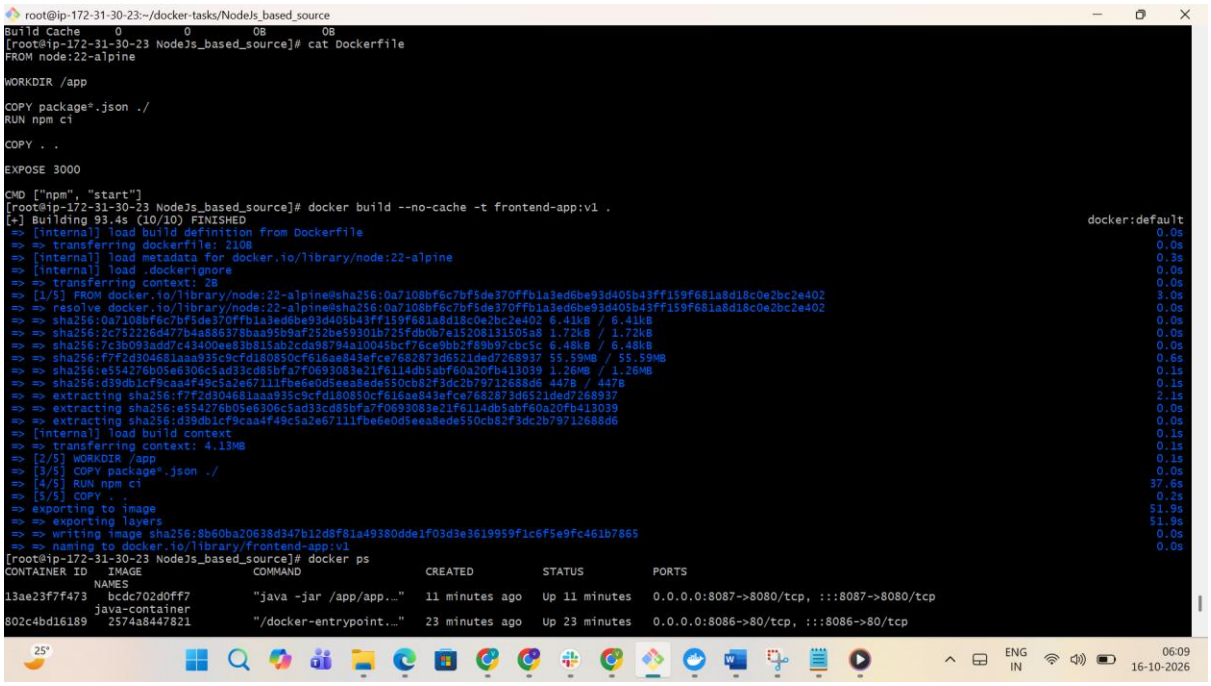
8. Tested the application using the correct URL and ensured that the application listened on 0.0.0.0 inside the container.
9. Logged in to Docker Hub.
10. Tagged the image:

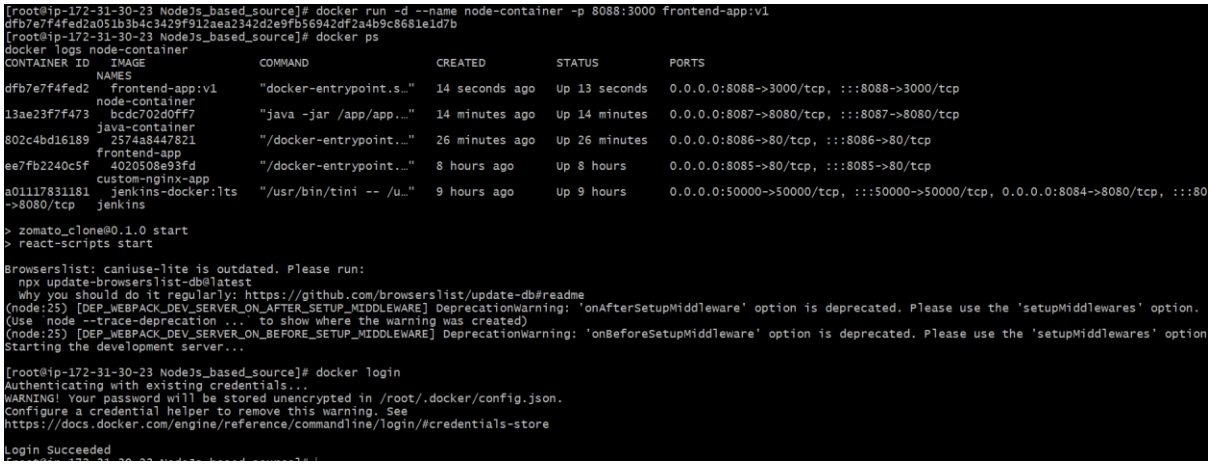
    docker tag frontend-app:v1 DOCKERHUB_USERNAME/node-app:v1

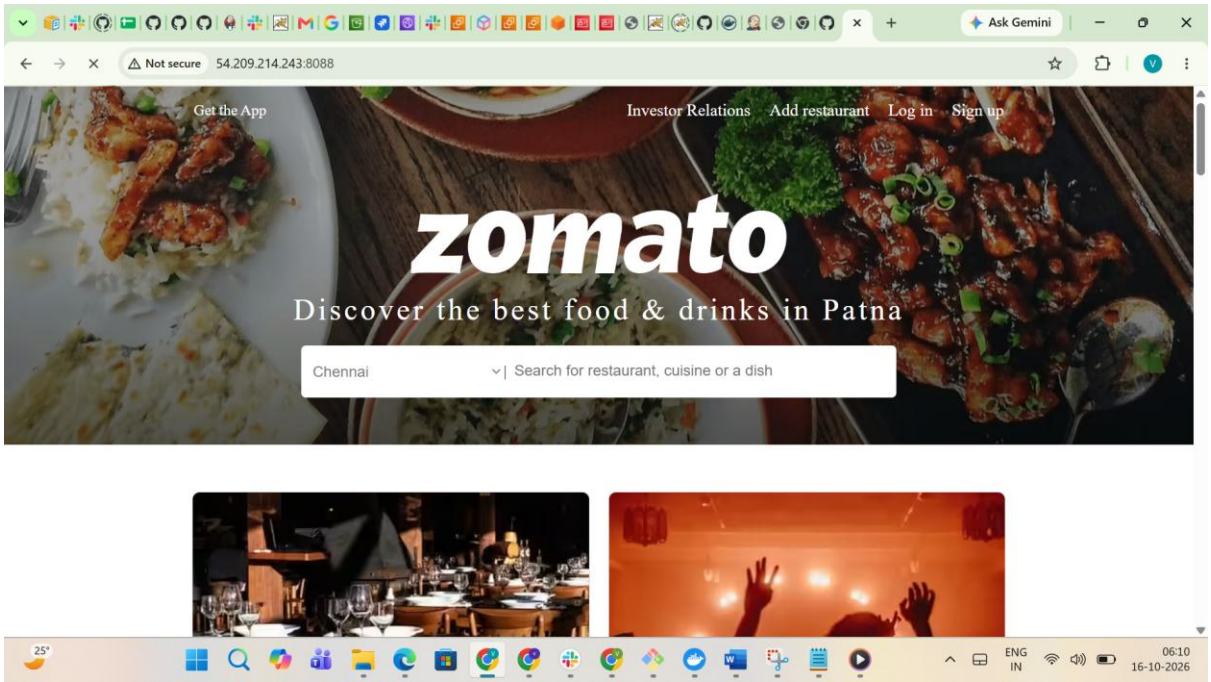
11. Pushed the image:

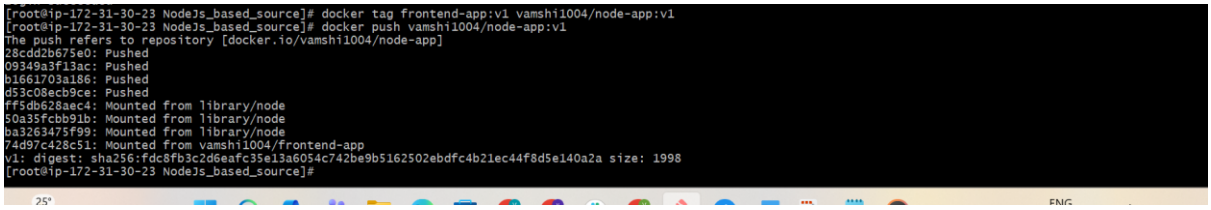
    docker push DOCKERHUB_USERNAME/node-app:v1

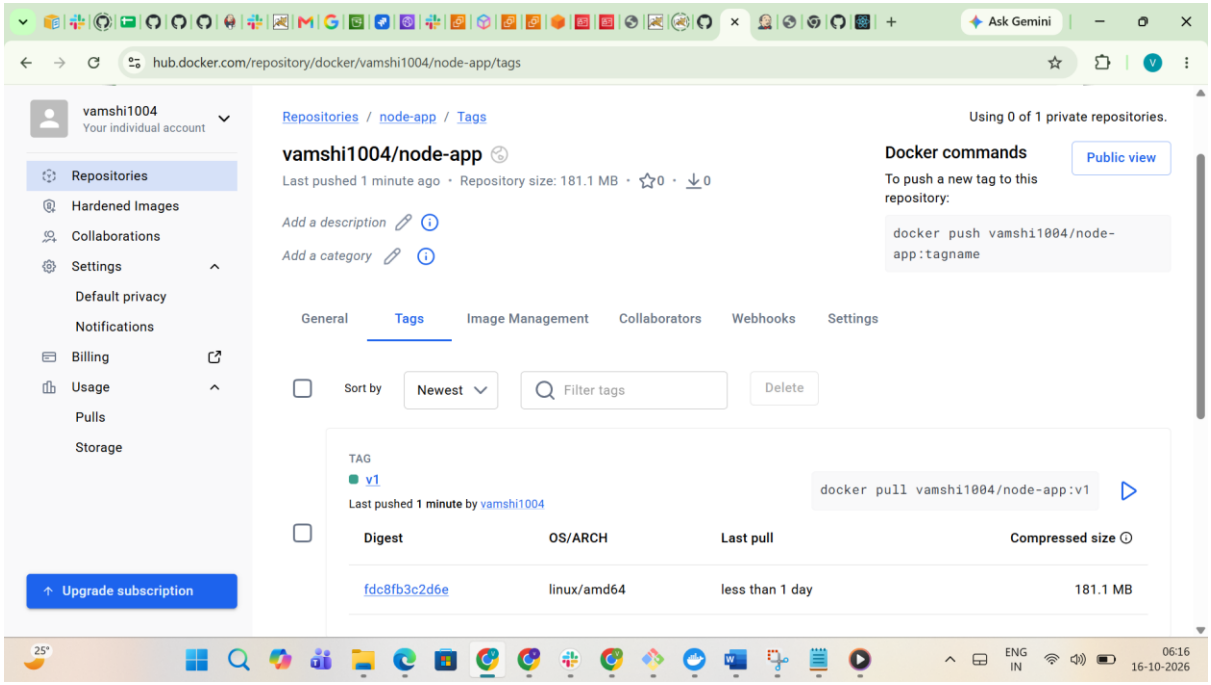
12. Verified that the v1 tag was available in Docker Hub.


root@ip-172-31-30-23:~/docker-tasks/NodeJs_based_source
Build Cache 0 0 0B 0B
[root@ip-172-31-30-23 NodeJs_based_source]# cat Dockerfile
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
[root@ip-172-31-30-23 NodeJs_based_source]# docker build --no-cache -t frontend-app:v1 .
[+] Building 93.4s (10/10) FINISHED docker:default
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 210B 0.0s
=> [internal] load metadata for docker.io/library/node:22-alpine 0.3s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/node:22-alpine@sha256:0a7108bf6c7bf5de370ffb1a3ed6be93d405b43ff159f681a8d18c0e2bc2e402 3.0s
=> => resolve docker.io/library/node:22-alpine@sha256:0a7108bf6c7bf5de370ffb1a3ed6be93d405b43ff159f681a8d18c0e2bc2e402 0.0s
=> => sha256:0a7108bf6c7bf5de370ffb1a3ed6be93d405b43ff159f681a8d18c0e2bc2e402 6.41kB / 6.41kB 0.0s
=> => sha256:2c752226d477b4a886378baa95b9af252be59301b725fdb0b7e1520813105a8 1.72kB / 1.72kB 0.0s
=> => sha256:7c3b093add7c43400ee83b815ab2cda98794a10045bcf76ce9bb2f89b97cbc5c 6.48kB / 6.48kB 0.0s
=> => sha256:f7f2d304681aaa935c9cfd180850cf616ae843efce7682873d6521ded7268937 55.59MB / 55.59MB 0.6s
=> => sha256:e554276b05e6306c5ad33cd85bfa7f0693083e21f6114db5abf60a20fb413039 1.26MB / 1.26MB 0.1s
=> => sha256:d39db1cf9caa4f49c5a2e67111fbe6e0d5eea8ede550cb82f3dc2b79712688d6 447B / 447B 0.1s
=> => extracting sha256:f7f2d304681aaa935c9cfd180850cf616ae843efce7682873d6521ded7268937 2.1s
=> => extracting sha256:e554276b05e6306c5ad33cd85bfa7f0693083e21f6114db5abf60a20fb413039 0.0s
=> => extracting sha256:d39db1cf9caa4f49c5a2e67111fbe6e0d5eea8ede550cb82f3dc2b79712688d6 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 4.13MB 0.1s
=> [2/5] WORKDIR /app 0.1s
=> [3/5] COPY package*.json ./ 0.0s
=> [4/5] RUN npm ci 37.6s
=> [5/5] COPY . . 0.2s
=> exporting to image 51.9s
=> => exporting layers 51.9s
=> => writing image sha256:8b60ba20638d347b12d8f81a49380dde1f03d3e3619959f1c6f5e9fc461b7865 0.0s
=> => naming to docker.io/library/frontend-app:v1 0.0s
[root@ip-172-31-30-23 NodeJs_based_source]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
13ae23f7f473 bcdc702d0ff7 "java -jar /app/app.…" 11 minutes ago Up 11 minutes 0.0.0.0:8087->8080/tcp, :::8087->8080/tcp java-container
802c4bd16189 2574a8447821 "/docker-entrypoint.…" 23 minutes ago Up 23 minutes 0.0.0.0:8086->80/tcp, :::8086->80/tcp
ENG IN 06:09 16-10-2026


[root@ip-172-31-30-23 NodeJs_based_source]# docker run -d --name node-container -p 8088:3000 frontend-app:v1
dfb7e7f4fed2a051b3b4c3429f912aea2342d2e9fb56942df2a4b9c8681e1d7b
[root@ip-172-31-30-23 NodeJs_based_source]# docker ps
docker logs node-container
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
dfb7e7f4fed2 frontend-app:v1 "docker-entrypoint.s…" 14 seconds ago Up 13 seconds 0.0.0.0:8088->3000/tcp, :::8088->3000/tcp node-container
13ae23f7f473 bcdc702d0ff7 "java -jar /app/app.…" 14 minutes ago Up 14 minutes 0.0.0.0:8087->8080/tcp, :::8087->8080/tcp java-container
802c4bd16189 2574a8447821 "/docker-entrypoint.…" 26 minutes ago Up 26 minutes 0.0.0.0:8086->80/tcp, :::8086->80/tcp frontend-app
ee7fb2240c5f 4020508e93fd "/docker-entrypoint.…" 8 hours ago Up 8 hours 0.0.0.0:8085->80/tcp, :::8085->80/tcp custom-nginx-app
a01117831181 jenkins-docker:lts "/usr/bin/tini -- /u…" 9 hours ago Up 9 hours 0.0.0.0:50000->50000/tcp, :::50000->50000/tcp, 0.0.0.0:8084->8080/tcp, :::8080->8080/tcp jenkins
> zomato_clone@0.1.0 start
> react-scripts start
Browserslist: caniuse-lite is outdated. Please run:
npx update-browserslist-db@latest
Why you should do it regularly: https://github.com/browserslist/update-db#readme
(node:25) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfterSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
(Use `node --trace-deprecation ...` to show where the warning was created)
(node:25) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBeforeSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
Starting the development server...
[root@ip-172-31-30-23 NodeJs_based_source]# docker login
Authenticating with existing credentials...
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store
Login Succeeded


Ask Gemini
Not secure 54.209.214.243:8088
Get the App
Investor Relations Add restaurant Log in Sign up
zomato
Discover the best food & drinks in Patna
Chennai
Search for restaurant, cuisine or a dish
ENG IN 06:10 16-10-2026





Question 9: Create a Docker Compose File to Set Up WordPress with MySQL

Implementation Steps

1. Created a project directory:

   mkdir -p /root/wordpress-mysql

   cd /root/wordpress-mysql

2. Created a file named docker-compose.yml.
3. Configured the MySQL service with the mysql:5.7 image, database name, username, and passwords.
4. Configured a named volume to persist MySQL data.
5. Configured the WordPress service using the wordpress:latest image.
6. Set the WordPress database host to db:3306, using the Compose service name for internal network communication.
7. Mapped host port 8000 to container port 80.
8. Configured the services to restart automatically.
9. Used the following Compose configuration:

   services:

```
db:
  image: mysql:5.7
  volumes:
    - db_data:/var/lib/mysql
  restart: always
  environment:
    MYSQL_ROOT_PASSWORD: somewordpress
    MYSQL_DATABASE: wordpress
    MYSQL_USER: wordpress
    MYSQL_PASSWORD: wordpress

wordpress:
  depends_on:
    - db
  image: wordpress:latest
  ports:
    - "8000:80"
  restart: always
  environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: wordpress
    WORDPRESS_DB_PASSWORD: wordpress
    WORDPRESS_DB_NAME: wordpress

volumes:
        db_data:
```

10. Validated the Compose configuration:

    docker compose config

11. Started the application:

    docker compose up -d

12. Checked the status of both services:

docker compose ps

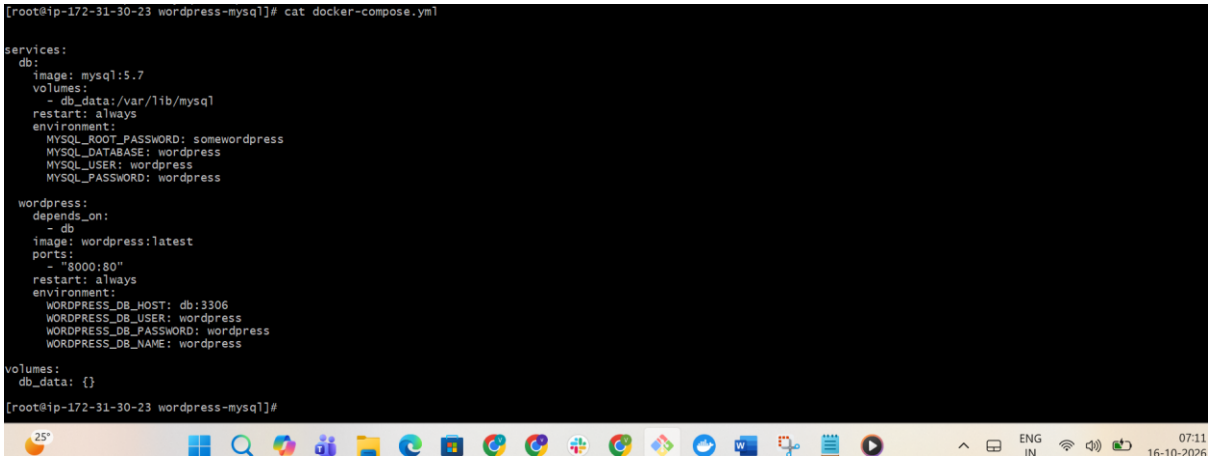
13. Reviewed the MySQL logs:

    docker compose logs --tail=100 db

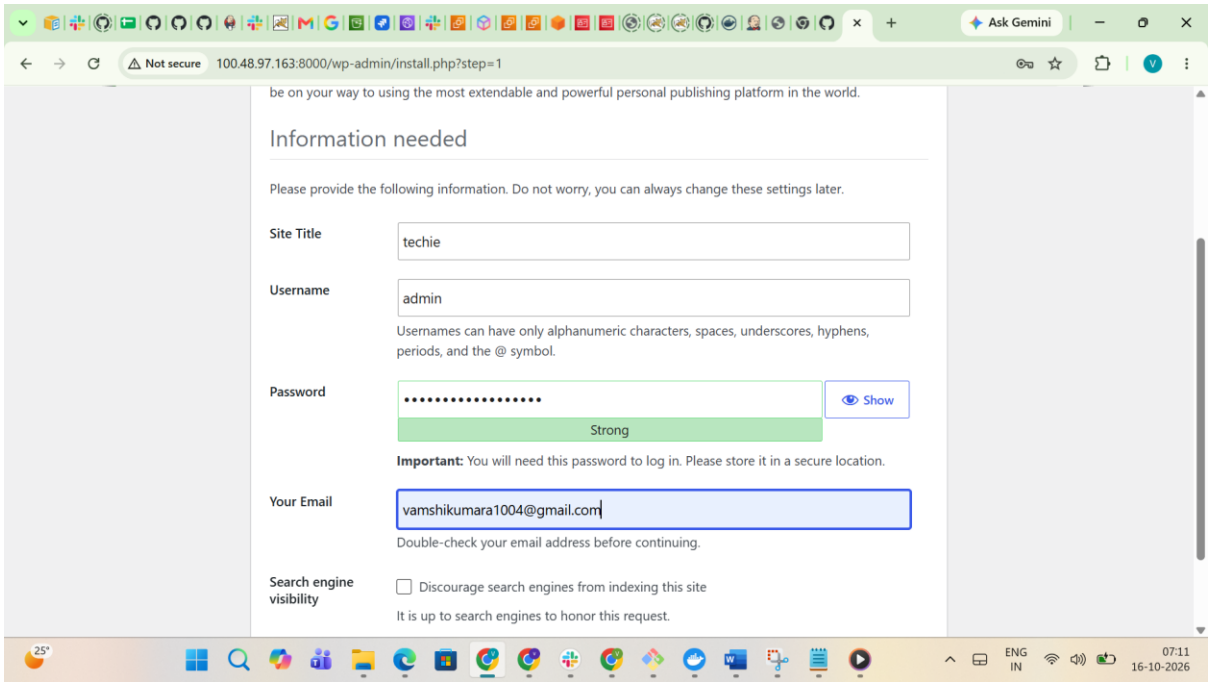
14. Verified that MySQL completed initialization and was ready for connections.

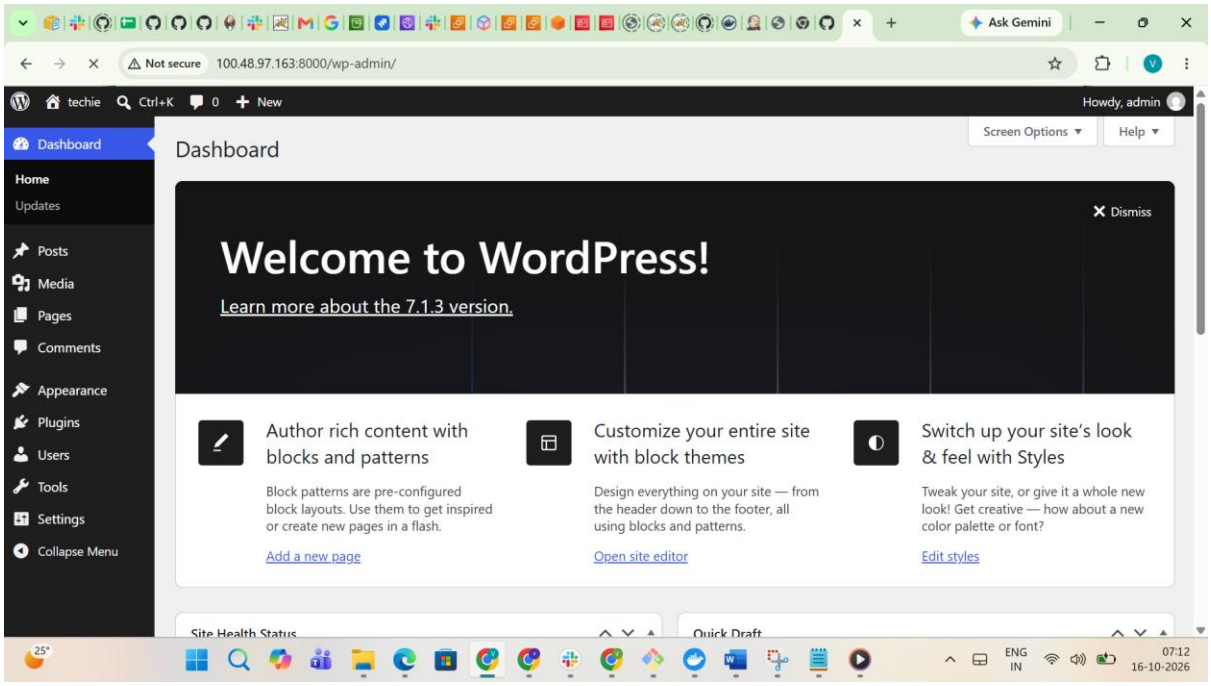
15. Opened the EC2 security group and allowed inbound TCP port 8000 from the required client IP.

16. Opened http://EC2_PUBLIC_IP:8000 in a browser.








Ask Gemini
Not secure
100.48.97.163:8000/wp-admin/
techie
Ctrl+K
0
New
Howdy, admin
Dashboard
Home
Updates
Posts
Media
Pages
Comments
Appearance
Plugins
Users
Tools
Settings
Collapse Menu
Dashboard
Screen Options
Help
Dismiss
Welcome to WordPress!
Learn more about the 7.1.3 version.
Author rich content with blocks and patterns
Block patterns are pre-configured block layouts. Use them to get inspired or create new pages in a flash.
Add a new page
Customize your entire site with block themes
Design everything on your site — from the header down to the footer, all using blocks and patterns.
Open site editor
Switch up your site's look & feel with Styles
Tweak your site, or give it a whole new look! Get creative — how about a new color palette or font?
Edit styles
Site Health Status
Quick Draft
25°
ENG
IN
07:12
16-10-2026